WORKBOOK

MediaTracker

An Android app in React Native and strict TypeScript that tracks the movies, series and anime I watch.

TMDB supplies what exists (search, series, seasons, episodes, movies and posters), and a SQLite database on the phone remembers what I have watched. The screens only show data and pass my taps on, while pure, tested functions calculate progress. The finished app shows a poster grid filtered by Watching, Done and Watchlist, with a progress badge on each poster, and a details page where I mark episodes as watched and see when the next episode airs.

Stack	React Native 0.87 (community CLI, no Expo) · strict TypeScript · React Navigation 7 (native stack) · op-sqlite · TMDB API
Checks	`tsc --noEmit` · ESLint · Prettier · Jest (logic only, better-sqlite3 for SQL tests)
Environment	Arch Linux · Neovim · Android SDK with the Android CLI (no Android Studio) · Pixel over USB
Scope	One user, one phone, local data. No accounts, sync, notifications, offline show info, date sections, Continue button, screen tests or release APK.
Size	9 milestones, 52 steps

How to use this workbook

Every milestone has a briefing that explains the ideas, then steps. Every step has the same parts, always in this order:

Part	What it gives you	How to use it
Task	What to do, in one or two sentences.	Read it first so you know where the step ends.
What this means	The problem the step solves, what goes in, what comes out, and what you will see when it works.	Read it before you open Neovim. If a word is new, look it up in the briefing or a cheat sheet.
Shape	The outside of the solution: file path, names, parameter and return types, and what happens with bad input.	Use the exact names. Later steps depend on them.
Hint 1	Direction: which idea or technique solves the step.	Read it, close the workbook, try.
Hint 2	Tools: the exact functions or APIs, a docs link, and search terms.	Only when Hint 1 was not enough. Read the docs page it points to.
Hint 3	Plan: the solution as numbered steps in plain words.	Last resort. Translate each line into code yourself.
Check	A command or an observation with the exact expected result.	The step is done only when you see exactly that.
Common mistake	The trap most people fall into here.	Read it after you finish, or when you are stuck.

Rules

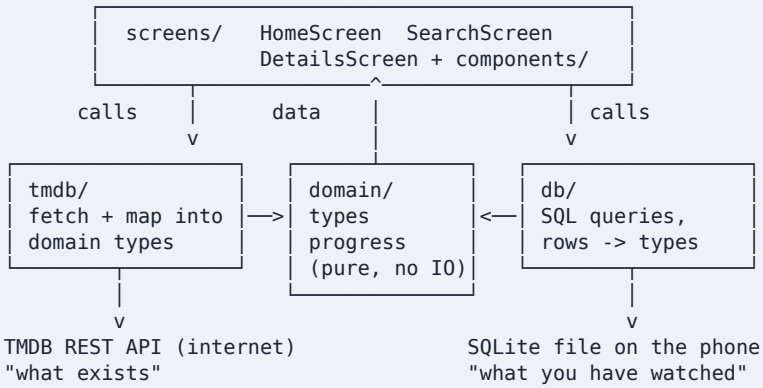
1. No step is skipped. Later steps use the names and files that earlier steps create.
2. No code is copied from anywhere: not from this workbook, not from docs examples, not from an AI. Read examples to understand them, then close them and write your own.
3. After every step, `npm run check` must pass (type check, linter and tests). From step 2.4 on, that one command runs all three.
4. Progress is tracked in `README.md` (it comes with this workbook, step 2.2), one line per milestone. Tick a milestone there only when every box in its "Done when" list is ticked and committed. The steps themselves stay in this PDF.
5. Every command runs from the repository root, the `MediaTracker` folder. The only exception is Milestone 1, which runs before the repository exists.
6. When a step adds something temporary, it also says when to remove it: usually at the end of the same step, sometimes in a named later step. Do not forget it, and do not remove it earlier.

Reference

The core rule

> **TMDB tells the app what exists, SQLite remembers what you watched, and the screens only show data and pass your taps on: they never calculate progress themselves.**

Architecture



Commands

Run every command from the repository root.

Command	What it does
`npm install`	Install dependencies.
`npm start`	Start Metro (keep it running in its own terminal).
`npm run android`	Build, install and open the app on the phone.
`npm run check`	Type check, lint and tests in one go.
`npm run typecheck`	Type check only.
`npm run lint`	Lint only.
`npm test`	Run all tests.
`adb devices`	Check that the phone is connected.
`adb logcat ReactNativeJS:V *:S`	Show the app's log output.
`android sdk list`	List installed Android SDK packages.

Structure

The same layout as in `README.md`, one comment per folder.

```
MediaTracker/
├── android/        generated Android project, built by Gradle
├── assets/         bundled images (the TMDB logo)
├── docs/
│   └── MediaTracker-Workbook.pdf
└── src/
    ├── config/     TMDB token (git-ignored) and its example file
    ├── domain/     types and pure progress logic, no I/O
    ├── tmdb/       TMDB client, response types, mapping and fixtures
    ├── db/         SQLite access, schema and queries
    ├── hooks/      reusable React hooks
    ├── navigation/ stack navigator and route types
    ├── components/ small reusable UI pieces
    └── screens/    Home, Search and Details
```

Files in each folder (when finished)

Folder	Files (when finished)
repository root	`App.tsx` (providers and navigator), `index.js` (generated), `package.json`, `tsconfig.json`, `README.md`
`android/`	generated; you only edit `MainActivity.kt` and `AndroidManifest.xml` (step 6.1)
`assets/`	`tmdb-logo.png`
`docs/`	`MediaTracker-Workbook.pdf`
`src/config/`	`secrets.ts` (git-ignored), `secrets.example.ts`
`src/domain/`	`types.ts`, `progress.ts` + `progress.test.ts`, `newTitle.ts` + `newTitle.test.ts`
`src/tmdb/`	`__fixtures__/` (four JSON files), `types.ts`, `map.ts` + `map.test.ts`, `images.ts` + `images.test.ts`, `client.ts` + `client.test.ts`, `api.ts`
`src/db/`	`database.ts`, `testDb.ts` + `testDb.test.ts`, `appDb.ts`, `rows.ts`, `schema.ts` + `schema.test.ts`, `titles.ts` + `titles.test.ts`, `episodes.ts` + `episodes.test.ts`
`src/hooks/`	`useDebouncedValue.ts`
`src/navigation/`	`types.ts`, `RootNavigator.tsx`
`src/components/`	`StatusFilter.tsx`, `ProgressBadge.tsx`, `PosterCard.tsx`, `SearchResultRow.tsx`, `EpisodeRow.tsx`
`src/screens/`	`HomeScreen.tsx`, `SearchScreen.tsx`, `DetailsScreen.tsx`

MILESTONE 1

A clean Android toolchain

Goal. Install everything needed to build Android apps from the terminal, and prove that your computer can talk to your phone.

You will learn. JDK · Android SDK · Android CLI (`android sdk`) · environment variables · PATH · adb · udev rules

Before you start.

- React Native environment setup (read the Linux + Android part, skip Android Studio): https://reactnative.dev/docs/set-up-your-environment
- Android CLI overview and its `sdk` command: https://developer.android.com/tools/agents/android-cli and https://developer.android.com/tools/agents/android-cli/commands/sdk
- sdkmanager page (now marked deprecated, but it still describes the folder layout): https://developer.android.com/tools/sdkmanager
- Android environment variables: https://developer.android.com/tools/variables
- Arch Wiki, Android (the parts about udev and adb): https://wiki.archlinux.org/title/Android
- This milestone runs before the repository exists, so run its commands from your home folder.

Briefing

The idea in plain words

A React Native Android app is built by three groups of tools working together. **Node.js** runs all the JavaScript tooling: the React Native CLI, Metro (the bundler that serves your code to the phone), Jest, ESLint and Prettier. The **JDK** (Java Development Kit) runs **Gradle**, the build tool that turns the `android/` folder into an installable app. The **Android SDK** gives Gradle the Android libraries to compile against, plus **adb**, the program that installs apps on your phone and carries log messages back.

Android Studio normally downloads the SDK for you. You are skipping it, so you use Google's **command-line tools** instead. That package used to be driven by `sdkmanager`. Google has deprecated it: its docs page now points to the **Android CLI**, a program called `android`, whose `android sdk` command installs, lists, updates and removes SDK packages. The `sdkmanager` file is still in the package, but when you run it, it prints a deprecation warning and hands over to the Android CLI. This workbook uses `android sdk` directly.

You put the whole SDK in a fresh folder in your home directory (`~/Android/Sdk`), so everything belongs to your user and nothing after the first `pacman` commands needs root. If you have an older SDK somewhere else, leave it alone for now. `ANDROID_HOME` tells Gradle and React Native which SDK to use, and a small file `~/.androidrc` tells the Android CLI the same thing. You can delete the old SDK later by hand once everything works.

This milestone only installs `platform-tools` (which contains adb). The other SDK packages (the Android platform, build-tools and the NDK) must match exact version numbers that are written inside the project. The project does not exist yet, so you install them in step 2.3, reading the versions from the project itself. That way the workbook can never be out of date with your template.

Where it fits

```
Node.js ─── React Native CLI, Metro, Jest, ESLint, Prettier
JDK 17  ─── Gradle (builds android/)
               └── Android SDK  ~/Android/Sdk
                     ├── cmdline-tools/latest   (android, the Android CLI)
                     ├── platform-tools         (adb)        ← this milestone
                     └── platform, build-tools, NDK          ← step 2.3
                               adb ── USB ── your Pixel
```

New words

Term	Meaning
JDK	Java Development Kit. Gradle needs it to run. React Native recommends version 17.
Gradle	The build tool for Android projects. It reads `android/build.gradle` files and produces the app.
Android SDK	The folder of Android tools and libraries that Gradle builds against.
Android CLI	Google's command-line program `android`. Its `android sdk` command manages SDK packages.
sdkmanager	The older SDK package tool. Deprecated, and now forwards to the Android CLI.
SDK package path	The name of one SDK package, like `platform-tools` or `build-tools/37.0.0`.
adb	Android Debug Bridge. Installs apps on the phone, forwards ports and shows logs.
Environment variable	A named value that every program started from your shell can read, like `ANDROID_HOME`.
PATH	The list of folders your shell searches when you type a command name.
udev rule	A Linux rule that decides which users may open a USB device. Needed so adb can reach the phone.

Steps

1.1 Install Node, JDK 17 and the USB rules

Task. Install Node.js 22.11 or newer, npm, JDK 17, `unzip` and the Android udev rules with pacman, then make JDK 17 the default Java and add yourself to the `adbusers` group.

What this means. Node runs every JavaScript tool in this project, and React Native needs at least version 22.11. Arch can have several JDKs installed at once, and a helper called `archlinux-java` picks which one runs when you type `java`. React Native recommends JDK 17 and warns that newer JDKs may cause problems, so you pick 17. The `android-udev` package adds rules that let members of the `adbusers` group open Android phones over USB. When this step works, `java -version` reports 17 and your user is in the `adbusers` group.

Shape.

- Packages: `nodejs`, `npm`, `jdk17-openjdk`, `android-udev`, `unzip`.
- Default Java environment: `java-17-openjdk`.
- Your user is a member of the group `adbusers`.
- No files to write in this step.

Hint 1, direction. Arch handles all of this with packages and one helper command for the Java default. A new group only takes effect in a new login session.

Hint 2, tools. `pacman -S`, `archlinux-java status`, `archlinux-java set` (needs sudo), `usermod -aG` or `gpasswd -a`. Docs: https://wiki.archlinux.org/title/Java, https://wiki.archlinux.org/title/Android, https://github.com/M0Rf30/android-udev-rules. Search: "archlinux-java set default jdk", "android-udev adbusers group".

Hint 3, plan. (1) Install the five packages with pacman. (2) Run `archlinux-java status` and read the names it lists. (3) Set the default to `java-17-openjdk`. (4) Add your user to `adbusers`. (5) Log out and back in (or reboot) so the new group applies.

Check. `node --version` prints v22.11.0 or higher. `java -version` prints a first line containing `"17.`. `groups` lists `adbusers`.

Common mistake. Leaving a newer JDK as the default. React Native's setup page warns that higher JDK versions may cause problems, and build errors from a wrong JDK are hard to read.

1.2 Put the command-line tools in a fresh SDK folder

Task. Download the Android command-line tools for Linux and place them so that this folder exists: `~/Android/Sdk/cmdline-tools/latest/bin/`.

What this means. Google's instructions require one exact layout: the contents of the zip go into `<sdk root>/cmdline-tools/latest/`. The zip itself contains a folder called `cmdline-tools` with `bin/`, `lib/`, `NOTICE.txt` and `source.properties` inside, so unpacking it directly gives the wrong layout. You move those contents one level down into a folder named `latest`. Inside `latest` you will find the Android CLI (`android`) and the old `sdkmanager`. When this works, the tools run and report a version.

Shape.

- SDK root: `~/Android/Sdk`.
- Final layout: `~/Android/Sdk/cmdline-tools/latest/` containing `bin/`, `lib/`, `NOTICE.txt` and `source.properties`.
- Download: the file named `commandlinetools-linux-<number>_latest.zip` from the "Command line tools only" table.

Hint 1, direction. The folder layout is the whole point of this step. Picture where the files must end up, then work backwards.

Hint 2, tools. Download page: https://developer.android.com/studio#command-tools. Layout instructions: https://developer.android.com/tools/sdkmanager. Use `mkdir -p`, `unzip`, `mv`. Search: "cmdline-tools latest folder layout".

Hint 3, plan. (1) Create `~/Android/Sdk/cmdline-tools`. (2) Unzip the download into a temporary folder. (3) Move the unpacked `cmdline-tools` folder to `~/Android/Sdk/cmdline-tools/latest`. (4) Delete the temporary folder and the zip. (5) Find where the `android` program is inside `latest`.

Check. `~/Android/Sdk/cmdline-tools/latest/bin/sdkmanager --version` prints a deprecation warning that mentions the Android CLI, then a version line ending in `(Android CLI)`. `find ~/Android/Sdk/cmdline-tools/latest -maxdepth 2 -name android` prints one path. Note its folder: you add that folder to `PATH` in step 1.3.

Common mistake. Unpacking straight into `~/Android/Sdk/`, which gives `cmdline-tools/bin` without `latest`.

1.3 Set the environment variables and the Android CLI default

Task. In your shell's config file, set `ANDROID_HOME` and `JAVA_HOME`, and add the folder with `android` and the `platform-tools` folder to `PATH`. Then create `~/.androidrc` so the Android CLI uses the same SDK.

What this means. Gradle and the React Native CLI read `ANDROID_HOME` to find the SDK, and Gradle reads `JAVA_HOME` to find the JDK. Adding two SDK folders to `PATH` lets you type `android` and `adb` from anywhere. The Android CLI has its own setting: its docs say you can put `--sdk=<path>` in `~/.androidrc` to make that SDK the default. A shell reads its config file only when it starts, so the change shows up in new terminals, not in the one you have open. When it works, a new terminal finds `android` by name and the CLI reports your SDK path.

Shape.

- `ANDROID_HOME` = `$HOME/Android/Sdk`
- `JAVA_HOME` = `/usr/lib/jvm/java-17-openjdk`
- Added to `PATH`: the folder that contains `android` (from step 1.2, normally `$ANDROID_HOME/cmdline-tools/latest/bin`) and `$ANDROID_HOME/platform-tools`.
- `~/.androidrc`: one line, `--sdk=/home/<you>/Android/Sdk` (a full path, no `~` or `$HOME`).
- Shell config file: the one for the shell you use (`~/.zshrc`, `~/.bashrc`, or `~/.config/fish/config.fish`).

Hint 1, direction. This is about exporting variables at shell start-up, plus one small config file. Add to `PATH`, never replace it.

Hint 2, tools. `export NAME=value` in bash and zsh, `set -gx NAME value` in fish. `echo $PATH` shows the result. `android info` shows which SDK the CLI uses. Docs: https://developer.android.com/tools/variables, https://developer.android.com/tools/agents/android-cli. Search: "export PATH append zsh", "fish set -gx PATH".

Hint 3, plan. (1) Open your shell config in Neovim. (2) Set `ANDROID_HOME`. (3) Set `JAVA_HOME`. (4) Append the two folders to `PATH`, using `ANDROID_HOME` inside the paths. (5) Write `~/.androidrc`. (6) Save, then open a new terminal.

Check. In a new terminal: `echo $ANDROID_HOME` prints `/home/<you>/Android/Sdk`, `which android` prints a path inside `~/Android/Sdk/cmdline-tools/latest`, and `android info` mentions `/home/<you>/Android/Sdk` as the SDK.

Common mistake. Testing in the terminal you edited from. It still has the old values.

1.4 Install platform-tools with the Android CLI

Task. Install the `platform-tools` package with `android sdk install`, then list what is installed.

What this means. `platform-tools` contains adb, which you need to talk to the phone in the next step. The Android CLI names packages with slash paths such as `platform-tools`, `platforms/android-37` or `build-tools/37.0.0`. Semicolon names from older guides (`platforms;android-37`) are still accepted. Google's SDK packages come with licenses. The Android CLI docs do not describe how it asks for them, so if the command shows a license, read it and accept it. When this works, adb exists and reports its version.

Shape.

- Package: `platform-tools`.
- Nothing else yet. The project-specific packages come in step 2.3.

Hint 1, direction. One install command, one list command.

Hint 2, tools. `android sdk install <package>`, `android sdk list` (installed only) and `android sdk list --all` (also what is available). Docs: https://developer.android.com/tools/agents/android-cli/commands/sdk_install and https://developer.android.com/tools/agents/android-cli/commands/sdk_list. Search: "android cli sdk install".

Hint 3, plan. (1) Run the install command for `platform-tools`. (2) Accept the license if one is shown. (3) Run `android sdk list`. (4) Run `adb version`.

Check. `android sdk list` shows `platform-tools` under "Installed packages". `which adb` prints a path inside `~/Android/Sdk/platform-tools`, and `adb version` prints a line starting with `Android Debug Bridge version`.

Common mistake. Getting `adb` from a different place (for example Arch's `android-tools` package) without noticing. `which adb` tells you which one runs.

1.5 Connect your phone

Task. Turn on Developer options and USB debugging on your Pixel, plug it in, and allow this computer.

What this means. Phones refuse to install apps over USB until you turn on USB debugging. The first time adb connects, the phone shows a prompt with your computer's key fingerprint. You allow it, and the phone remembers the key. `adb devices` shows one of three states: `device` (ready), `unauthorized` (prompt not accepted yet) or `offline` (connection problem). When it works, you see your phone's serial number followed by `device`.

Shape.

- On the phone: Settings → About phone → tap "Build number" seven times. Then find Developer options under Settings → System (Google's page lists it under System → Advanced on some Android versions) and turn on USB debugging.
- Exactly one device connected.

Hint 1, direction. Three things must line up: the phone allows debugging, Linux allows your user to open the USB device (step 1.1), and the phone trusts your computer.

Hint 2, tools. `adb devices`, `adb kill-server`. Docs: https://developer.android.com/studio/debug/dev-options and https://reactnative.dev/docs/running-on-device. Search: "adb devices unauthorized linux", "pixel enable usb debugging".

Hint 3, plan. (1) Turn on Developer options. (2) Turn on USB debugging. (3) Plug in the cable. (4) Run `adb devices`. (5) Accept the prompt on the phone and tick "Always allow". (6) Run `adb devices` again.

Check. `adb devices` prints `List of devices attached` and one line with a serial number followed by `device`.

Common mistake. Dismissing the phone prompt, which leaves the state at `unauthorized`. Unplug, run `adb kill-server`, plug in again and watch the phone.

Stuck?

If you see	Look at
`android: command not found`	Step 1.3: you are in an old terminal, or the `PATH` entry is not the folder that contains `android`.
`android info` shows a different SDK	`~/.androidrc` (step 1.3): full path, no `~`.
`no permissions (user in plugdev group...)` in `adb devices`	Step 1.1: you are not in `adbusers`, or you did not log out and in again.
`unauthorized`	Step 1.5: the prompt on the phone. Run `adb kill-server` and reconnect.
`java -version` shows 21 or newer	Step 1.1: `archlinux-java set`.

Done when

- ☐ `java -version` shows 17.
- ☐ `android sdk list` shows `platform-tools` as installed.
- ☐ A new terminal finds `android` and `adb` by name, and `android info` shows `~/Android/Sdk`.
- ☐ `adb devices` shows your phone as `device`.
- ☐ Committed with a message that names the milestone. (The repository is created in step 2.1, so this commit happens in step 2.2 as `Milestone 1: toolchain ready`.)

Answer without looking

1.1. Why must the contents of the command-line tools zip end up in `cmdline-tools/latest` and not directly in `cmdline-tools`?

1.2. Why does this milestone install only `platform-tools`, and not the platform, build-tools and NDK?

1.3. You edit `~/.zshrc` and the terminal you edited from still cannot find `adb`. Why?

1.4. `adb devices` shows `unauthorized`. Is this a Linux problem or a phone problem, and what fixes it?

1.5. Why install JDK 17 instead of the newest JDK?

MILESTONE 2

Project and editor

Goal. Create the React Native project in strict TypeScript, run it on your phone, and make the type check, linter and tests pass from both the terminal and Neovim.

You will learn. React Native CLI · Metro · Gradle build · NDK and CMake · tsconfig strictness · npm scripts · language servers · Fast Refresh

Before you start.

- Get started without a framework: https://reactnative.dev/docs/getting-started-without-a-framework
- Running on device: https://reactnative.dev/docs/running-on-device
- Using TypeScript: https://reactnative.dev/docs/typescript
- TSConfig reference for noUncheckedIndexedAccess: https://www.typescriptlang.org/tsconfig/#noUncheckedIndexedAccess
- nvim-lspconfig: https://github.com/neovim/nvim-lspconfig · conform.nvim: https://github.com/stevearc/conform.nvim · mason.nvim: https://github.com/mason-org/mason.nvim

Briefing

The idea in plain words

A React Native app runs in two halves. The **native half** is a normal Android app in the `android/` folder. Gradle builds it, and it contains a JavaScript engine. The **JavaScript half** is your TypeScript code. During development, Metro bundles it and sends it to the phone over the USB connection, so after a change you save the file and the phone reloads within a second (this is **Fast Refresh**). You only rebuild the native half when you add a library with native code or change files in `android/`.

The template already gives you strict TypeScript, ESLint, Prettier and Jest. You will make TypeScript one notch stricter with `noUncheckedIndexedAccess`. That setting makes `array[0]` have the type "element or undefined", because an array can be empty. It forces you to handle the empty case, and it catches real bugs in code that reads TMDB lists.

Finally, you connect Neovim to the same tools through language servers. A **language server** is a background program the editor talks to. It reports errors while you type, offers completion and jumps to definitions. The goal is that what Neovim shows matches what `npm run check` says.

Where it fits

```
Neovim ── vtsls (types) ── eslint (lint) ── prettier (format on save)
   │
App.tsx, src/**   ──Metro──> USB ──> phone (JS engine runs your code)
android/          ──Gradle─> app file installed once per native change
npm run check  =  tsc --noEmit  →  eslint .  →  jest
```

New words

Term	Meaning
Metro	React Native's bundler. It collects your TypeScript files into one bundle and serves it to the phone.
Fast Refresh	Reloading your changed code in the running app without rebuilding it.
NDK	Native Development Kit. Lets Gradle compile C++ code for Android.
CMake	A build system for C++ that the NDK build uses.
compileSdkVersion	The Android API level the app is compiled against. It decides which `platforms/android-<n>` package you need.
npm script	A named command in `package.json` under `scripts`, run with `npm run <name>`.
Type check	Running the TypeScript compiler only to find type errors, without producing files.

Term	Meaning
Language server (LSP)	A background program an editor uses for errors, completion and navigation.
Formatter	A tool that rewrites code layout (spaces, line breaks) to one fixed style. Here: Prettier.

Steps

2.1 Create the project

Task. In the folder where you keep projects, create a new React Native project called `MediaTracker` with the community CLI. From now on, the `MediaTracker` folder is the repository root.

```
npx @react-native-community/cli@latest init MediaTracker
```

What this means. The CLI downloads the current React Native template, installs the npm packages and sets up a git repository with a first commit. The template is already TypeScript: the app's entry component is `App.tsx`. Nothing is built yet, this only creates files. When it works, you have a folder with `android/`, `App.tsx` and `package.json`, and git history with one commit.

Shape.

- Project name: `MediaTracker` (letters only, no spaces or dashes).
- On Linux the CLI does not ask about CocoaPods (that question only appears on macOS).

Hint 1, direction. The command above is the whole step. Afterwards, look around the generated files so you know what is there.

Hint 2, tools. Docs: https://reactnative.dev/docs/getting-started-without-a-framework. Look at `package.json` (scripts and dependencies) and `tsconfig.json`. Search: "react native community cli init".

Hint 3, plan. (1) `cd` to your projects folder. (2) Run the command. (3) `cd MediaTracker`. (4) Run `git log --oneline` and `ls`.

Check. `ls` shows `android`, `App.tsx`, `index.js`, `package.json` and `tsconfig.json`. `git log --oneline` shows one commit. In `package.json`, the `react-native` dependency starts with `0.87`.

Common mistake. Having an old global `react-native-cli` installed. The React Native docs say to remove it first (`npm uninstall -g react-native-cli @react-native-community/cli`) because it can cause unexpected issues.

2.2 Put the README and workbook in place and make the Milestone 1 commit

Task. Replace the template's `README.md` with the `README.md` that came with this workbook, put this PDF in `docs/`, tick Milestone 1, and commit as `Milestone 1: toolchain ready`.

What this means. The README is short on purpose: one line per milestone, plus the folder structure and the commands. The steps stay in this PDF. You tick a milestone in the README only when every box in its "Done when" list here is ticked and the work is committed. The README links to `docs/MediaTracker-Workbook.pdf`, so the PDF goes into the repository too, and the link works on GitHub.

Shape.

- File: `README.md` in the repository root (replace the template's file completely).
- File: `docs/MediaTracker-Workbook.pdf`.
- README: Milestone 1 ticked (`- [x]`), all others unticked.

Hint 1, direction. Copy two files, change one `[ ]` to `[x]`, commit.

Hint 2, tools. `mkdir -p`, `cp`, `git add`, `git commit -m`. Markdown task lists: https://docs.github.com/en/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax#task-lists.

Hint 3, plan. (1) Copy the workbook's `README.md` over the template's. (2) Create `docs/` and copy this PDF into it. (3) Tick Milestone 1. (4) Commit with the message above.

Check. `git log --oneline` shows two commits, the newest is `Milestone 1: toolchain ready`. `grep -c '\- \[x\]' README.md` prints `1`. `ls docs` shows `MediaTracker-Workbook.pdf`.

2.3 Install the project's SDK packages and build on the phone

Task. Read the Android platform, build-tools and NDK versions that the project asks for, install exactly those with the Android CLI, then build and run the app on your phone.

What this means. The template pins exact versions in `android/build.gradle`, in the `ext` block: `compileSdkVersion` (the Android platform to compile against), `buildToolsVersion` and `ndkVersion` (React Native compiles C++ during the build, so it needs the NDK). These numbers can be newer than the React Native setup page says: for 0.87 the template asks for platform 37 and build-tools 37.0.0, while the setup page still mentions 35 and 36.0.0. The project file is what Gradle actually uses, so it wins. The first build downloads Gradle and many libraries and can take ten minutes or more. Metro runs in one terminal and the build in another. When it works, the phone shows the template's welcome screen.

Shape.

- Read `compileSdkVersion`, `buildToolsVersion` and `ndkVersion` in `android/build.gradle`. (For React Native 0.87 they are `37`, `"37.0.0"` and `"27.1.12297006"`.)
- Packages: `platforms/android-<compileSdkVersion>`, `build-tools/<buildToolsVersion>`, `ndk/<ndkVersion>`.
- If the build stops and names a missing CMake version, install `cmake/<that version>` the same way.
- Terminal 1: `npm start`. Terminal 2: `npm run android`.

Hint 1, direction. The project file tells you the versions. The Android CLI installs them. The build tells you if anything else is missing.

Hint 2, tools. `grep -E "compileSdkVersion|buildToolsVersion|ndkVersion" android/build.gradle`, `android sdk install <package> <package> ...`, `android sdk list`. Docs: https://developer.android.com/tools/agents/android-cli/commands/sdk_install, https://reactnative.dev/docs/running-on-device. Search: "react native ndkVersion build.gradle", "android cli sdk install ndk".

Hint 3, plan. (1) Read the three values. (2) Install the three packages in one `android sdk install` command. (3) Check them with `android sdk list`. (4) Start Metro in terminal 1. (5) Run the Android build in terminal 2. (6) If it fails on CMake, install the version it names and build again.

Check. `android sdk list` shows the platform, build-tools and NDK with the versions from `build.gradle`. Terminal 2 prints `BUILD SUCCESSFUL`, and the phone opens the app and shows "Welcome to React Native".

Common mistake. Installing the versions from the React Native setup page (platform 35, build-tools 36.0.0) instead of the ones in `build.gradle`. Gradle then cannot find the platform it was told to compile against.

2.4 Make TypeScript stricter and add one check command

Task. Turn on `noUncheckedIndexedAccess`, delete the template's screen test, and add npm scripts so that `npm run check` runs the type check, the linter and the tests.

What this means. The template's `tsconfig.json` extends a shared React Native config that already turns on `strict`. You add one more setting on top. You also delete `__tests__/App.test.tsx`: it renders the whole app, and once the app uses native libraries (Milestones 5 and 6) it can no longer run in Jest. You chose to test logic, not screens. Until your first test exists in Milestone 3, Jest would fail with "No tests found", so the test script gets a flag that allows zero tests. When it works, one command checks everything and exits without errors.

Shape.

- `tsconfig.json`: add `"compilerOptions": { "noUncheckedIndexedAccess": true }` next to the existing `extends`.
- Delete: `__tests__/App.test.tsx` (and the `__tests__` folder if it is then empty).
- `package.json` scripts: `"typecheck": "tsc --noEmit"`, `"test": "jest --passWithNoTests"`, `"check": "npm run typecheck && npm run lint && npm test"`.

Hint 1, direction. Configuration only: one tsconfig key and three scripts. `&&` runs the next command only if the previous one succeeded.

Hint 2, tools. TSConfig docs: https://www.typescriptlang.org/tsconfig/#noUncheckedIndexedAccess. npm scripts: https://docs.npmjs.com/cli/using-npm/scripts. Search: "tsc --noEmit", "jest passWithNoTests".

Hint 3, plan. (1) Add the compiler option to `tsconfig.json`. (2) Delete the template test. (3) Add or change the three scripts. (4) Run `npm run check`.

Check. `npm run check` runs `tsc`, then `eslint`, then `jest`, prints `No tests found, exiting with code 0`, and ends with exit code 0 (`echo $?` prints `0`).

Common mistake. Putting `noUncheckedIndexedAccess` outside `compilerOptions`. TypeScript ignores it without a warning.

2.5 Connect Neovim to the same tools

Task. Set up a TypeScript language server, the ESLint language server and Prettier formatting on save in Neovim.

What this means. Right now you only see errors when you run `npm run check`. A language server shows them while you type, in the same file. You need three pieces: `vtsls` for TypeScript errors and completion, the `eslint` server for lint warnings, and `conform.nvim` running Prettier when you save. All three read the project's own config files (`tsconfig.json` and the ESLint and Prettier config files the template created), so the editor agrees with the terminal. When it works, a type error shows up in Neovim before you save.

Shape.

- Language servers (install with Mason): `vtsls`, `eslint-lsp`.
- Formatter (install with Mason): `prettier`.
- conform.nvim: Prettier for the filetypes `typescript` and `typescriptreact`, format on save.
- Your Neovim config lives in your dotfiles, not in this repository. Nothing in this step is committed here.

Hint 1, direction. Neovim needs a server per job and a formatter hook on save. Mason installs the programs, lspconfig and conform connect them.

Hint 2, tools. Neovim 0.11+: `vim.lsp.enable({ 'vtsls', 'eslint' })` with nvim-lspconfig installed. conform.nvim: `formatters_by_ft` and `format_on_save`. Docs: https://github.com/neovim/nvim-lspconfig, https://github.com/stevearc/conform.nvim, https://github.com/yioneko/vtsls. Search: "neovim vim.lsp.enable vtsls", "conform.nvim prettier format_on_save".

Hint 3, plan. (1) Install the three tools with `:Mason`. (2) Enable the two servers. (3) Configure conform for the two filetypes with format on save. (4) Restart Neovim and open `App.tsx`. (5) Run `:checkhealth vim.lsp` and look for both servers attached.

Check. In `App.tsx`, type `const n: number = 'x';`. Neovim marks it with "Type 'string' is not assignable to type 'number'". Delete the line. Add extra spaces inside a JSX tag and save: they disappear.

Common mistake. Starting Neovim outside the repository root, so the servers do not find `tsconfig.json` and report wrong errors.

2.6 Make the app yours

Task. Replace the template content in `App.tsx` with a screen that only shows the text "Media Tracker" centered, and watch Fast Refresh update the phone.

What this means. This proves the edit, save, reload loop that you will use for the rest of the workbook. `App.tsx` exports one component, a function that returns what the screen shows. The template wraps everything in a `SafeAreaProvider` (from `react-native-safe-area-context`, already installed). Keep that wrapper, because step 6.4 uses it again, and replace only the demo content inside it. Because you change only TypeScript, there is no rebuild: Metro sends the new code and the phone updates by itself. When it works, the text changes on the phone within a second or two of saving.

Shape.

- File: `App.tsx`.
- Default export: a function component `App(): React.JSX.Element`.
- Inside the template's `SafeAreaProvider`: one `View` that fills the screen and centers one `Text` with "Media Tracker".

Hint 1, direction. A component is a function that returns elements. Layout uses flexbox, and a view that fills the screen and centers its child needs three style properties.

Hint 2, tools. `View`, `Text`, `StyleSheet.create`, style keys `flex`, `justifyContent`, `alignItems`. Docs: https://reactnative.dev/docs/view, https://reactnative.dev/docs/flexbox. Search: "react native center view flexbox".

Hint 3, plan. (1) Delete the template's demo content (`AppContent`, `NewAppScreen`) and imports you no longer use, but keep `SafeAreaProvider`. (2) Return a `View` with a style that fills the screen and centers. (3) Put a `Text` inside. (4) Make the styles with `StyleSheet.create`. (5) Save and watch the phone. (6) Run `npm run check`.

Check. The phone shows only "Media Tracker" in the middle. Change it to "Media Tracker!" and save: the phone updates without `npm run android`. `npm run check` passes.

Common mistake. Leaving unused imports from the template. ESLint fails on them in `npm run check`.

Stuck?

If you see	Look at
`SDK location not found`	`ANDROID_HOME` in this terminal (step 1.3).
`NDK ... did not have a source.properties file`	The NDK download was interrupted. Install it again (step 2.3).
Gradle says a platform, build-tools or NDK version is missing	Compare `android sdk list` with the versions in `android/build.gradle` (step 2.3).
Red screen on the phone: "Unable to load script"	Is Metro running? Run `adb reverse tcp:8081 tcp:8081` and reload.
Errors that mention Java or class file versions	Wrong JDK (step 1.1), or `JAVA_HOME` points to another JDK.
Neovim shows no errors at all	`:checkhealth vim.lsp`, and whether you opened Neovim inside `MediaTracker`.
`npm run check` fails in ESLint on files you did not touch	Template leftovers. Read the file and line it names.

Done when

- The app runs on your phone and shows "Media Tracker".
- `npm run check` passes.
- Neovim shows type errors and formats on save.
- README: Milestone 2 ticked.
- Committed with a message that names the milestone (`Milestone 2: project and editor`).

Answer without looking

2.1. You change text in `App.tsx`. Why don't you need `npm run android` again?

2.2. What does `noUncheckedIndexedAccess` change about the type of `list[0]`, and why is that useful for TMDB data?

2.3. Why was the template's `App.test.tsx` deleted instead of kept?

2.4. What is the job of Metro, and what is the job of Gradle?

MILESTONE 3

Your own types and progress logic

Goal. Describe the app's data with your own TypeScript types and write the tested, pure functions that turn watched episodes into a percentage, a label and a badge.

You will learn. type aliases · union types · discriminated unions · readonly arrays · pure functions · Jest: describe, it, expect

Before you start.

- TypeScript handbook, Everyday Types: https://www.typescriptlang.org/docs/handbook/2/everyday-types.html
- Narrowing and discriminated unions: https://www.typescriptlang.org/docs/handbook/2/narrowing.html
- Jest, Getting Started and Using Matchers: https://jestjs.io/docs/getting-started and https://jestjs.io/docs/using-matchers
- Cheat sheets B (TypeScript) and C (Jest) at the back of this workbook.

Briefing

The idea in plain words

Before you talk to TMDB or a database, you decide what the app's own data looks like. TMDB has its own field names (`first_air_date`, `poster_path`, and so on) and SQLite will have column names. Neither of those should leak into your screens. The `domain` folder holds the types that the rest of the app agrees on, plus functions that only calculate. A **pure function** gets everything it needs through its parameters, reads no files, calls no network and returns a value. The same input always gives the same output, which makes it easy to test.

Three kinds of type do most of the work here. A **union type** lists the allowed values, like `'watching' | 'done' | 'watchlist'`, so a typo becomes a compile error. An **object type** describes a shape, like a tracked title with a name and a status. A **discriminated union** is a union of object types that share one field with a different fixed value in each, like `kind`. When code checks that field, TypeScript knows which shape it has. You will use one for the poster badge: either a check mark, a percentage, or nothing.

Anime is not its own media type in this app. TMDB lists anime as normal TV series, so a title is either a `'movie'` or a `'tv'` show. Progress for a series is counted in episodes. Specials (TMDB's "season 0") are not counted, because most people never watch them and they would stop a series from ever reaching 100%.

Where it fits

```
src/domain/types.ts      ← the shapes everyone uses
src/domain/progress.ts   ← pure functions: percent, label, badge
       ^          ^           ^
     tmdb/       db/        screens/   (later milestones)
```

New words

Term	Meaning
Type alias	A name for a type: `type Status = ...`.
Union type	A type that is one of several options, written with `|`.
Discriminated union	A union of object types that each have the same field (here `kind`) with a different literal value.
Literal type	A type with exactly one value, like `'tv'` or `0`.
`null`	In this project, "there is no value". A poster path can be `null` when TMDB has no poster.
Pure function	A function whose result depends only on its parameters, with no side effects.

Term	Meaning
Test case	One `it(...)` block in Jest: one input, one expected output.
Episode reference	A season number plus an episode number, like S2 E5.

Steps

3.1 The small building blocks

Task. Create `src/domain/types.ts` with the types `MediaType`, `Status`, `EpisodeRef`, `Episode` and `SeasonSummary`.

What this means. These are the smallest pieces the other types are made of. `MediaType` and `Status` are unions of string literals. `EpisodeRef` points at one episode by season and episode number. `Episode` adds what you show in a list (name and air date). `SeasonSummary` is one row in a season picker. Nothing runs yet. You will know it works when the type check passes and a wrong value is rejected.

Shape.

- File: `src/domain/types.ts`. Export every type.
- `MediaType` = `'movie' | 'tv'`
- `Status` = `'watching' | 'done' | 'watchlist'`
- `EpisodeRef` = `{ seasonNumber: number; episodeNumber: number }`
- `Episode` = `EpisodeRef` plus `name: string` and `airDate: string | null` (format `YYYY-MM-DD`).
- `SeasonSummary` = `{ seasonNumber: number; name: string; episodeCount: number }`

Hint 1, direction. Type aliases with string literal unions and object types. One type can build on another.

Hint 2, tools. `type`, `|`, `&` (intersection) or repeating the fields. Docs: https://www.typescriptlang.org/docs/handbook/2/everyday-types.html. Search: "typescript string literal union", "typescript intersection type".

Hint 3, plan. (1) Write the two unions. (2) Write `EpisodeRef`. (3) Write `Episode` by combining `EpisodeRef` with the extra fields. (4) Write `SeasonSummary`. (5) Export all five.

Check. `npm run typecheck` passes. Temporarily add `const s: Status = 'paused';` at the bottom: `tsc` reports that `'paused'` is not assignable to type `Status`. Remove the line.

3.2 The title and details types

Task. Add `TrackedTitle`, `SearchResult`, `NextEpisode`, `TvDetails` and `MovieDetails` to `src/domain/types.ts`.

What this means. `TrackedTitle` is one row in your grid: something you added, with its status and how many episodes you have watched. `SearchResult` is one row in the search list. `TvDetails` and `MovieDetails` are what the details page shows, built from TMDB data in Milestone 4. Times are stored as numbers (milliseconds since 1970, the value `Date.now()` returns) because they are easy to compare and sort. When it works, the type check passes and the file holds every shape the app uses.

Shape.

- `TrackedTitle`: `id: number` (local database id), `tmdbId: number`, `mediaType: MediaType`, `name: string`, `posterPath: string | null`, `totalEpisodes: number`, `watchedCount: number`, `status: Status`, `addedAt: number`, `lastWatchedAt: number | null`.
- `SearchResult`: `tmdbId: number`, `mediaType: MediaType`, `name: string`, `year: string | null`, `posterPath: string | null`.
- `NextEpisode`: `EpisodeRef` plus `airDate: string`.

- `TvDetails`: `tmdbId`, `name`, `posterPath`, `overview: string`, `seasons: SeasonSummary[]`, `totalEpisodes: number`, `nextEpisode: NextEpisode | null`.
- `MovieDetails`: `tmdbId`, `name`, `posterPath`, `overview: string`, `runtimeMinutes: number | null`, `releaseDate: string | null`.

Hint 1, direction. Same tools as step 3.1. Decide for each field whether it can be missing, and write `| null` only then.

Hint 2, tools. Object types, `| null`. Search: "typescript optional vs null", "typescript object type alias".

Hint 3, plan. (1) Write `TrackedTitle` field by field from the list. (2) Write `SearchResult`. (3) Write `NextEpisode` by reusing `EpisodeRef`. (4) Write the two details types.

Check. `npm run typecheck` passes. In Neovim, hover over `TrackedTitle` where it is defined: the popup lists 10 fields.

Common mistake. Using optional fields (`posterPath?: string`) instead of `string | null`. Then "missing" has two spellings, `undefined` and absent, and the database will give you `null`.

3.3 Progress as a percentage

Task. Write `progressPercent` and its tests.

What this means. The badge on each poster shows how much of a series you have watched. The function gets the number of watched episodes and the total, and returns a whole number from 0 to 100. Two inputs are dangerous: a total of 0 (division by zero gives `NaN` or `Infinity`) and more watched than total (possible if TMDB later corrects an episode count). Both must give a sensible number. When it works, five tests pass.

Shape.

- File: `src/domain/progress.ts`. Export `progressPercent(watched: number, total: number): number`.
- Round to the nearest whole number.
- If `total` is 0 or less, return 0. Never return more than 100.
- Tests in `src/domain/progress.test.ts`.

Hint 1, direction. Guard clause first, then the calculation, then a limit on the result.

Hint 2, tools. `Math.round`, `Math.min`. Jest: `describe`, `it`, `expect(...).toBe(...)`. Docs: https://jestjs.io/docs/using-matchers. Search: "javascript Math.round percentage", "jest toBe".

Hint 3, plan. (1) If total is 0 or less, return 0. (2) Divide watched by total and multiply by 100. (3) Round. (4) Return the smaller of that and 100. (5) Write one `it` per test case below.

Check. `npm test` passes these cases: (5, 20) → 25. (1, 3) → 33. (2, 3) → 67. (0, 0) → 0. (30, 24) → 100.

Common mistake. Rounding before multiplying by 100, which turns 1/3 into 0.

3.4 The latest episode and its label

Task. Write `latestEpisode` and `progressLabel`, with tests.

What this means. The details page shows how far you are, like "S2 E5". "How far" means the highest episode you marked, ordered by season first and episode second. Order of marking does not matter: if you watched S2 E3 and later went back to S1 E10, you are still at S2 E3. `latestEpisode` finds that episode in an unsorted list, and `progressLabel` turns it into text. An empty list means you have not started. When it works, five tests pass.

Shape.

- File: `src/domain/progress.ts`.

- `latestEpisode(watched: readonly EpisodeRef[]): EpisodeRef | null`. Empty list → `null`. Do not change the input array.
- `progressLabel(latest: EpisodeRef | null): string`. `null` → `"Not started"`, otherwise `"S<season> E<episode>"`.

Hint 1, direction. Find a maximum with a comparison that looks at two fields in order. `readonly` stops you from sorting in place by accident.

Hint 2, tools. A loop or `Array.prototype.reduce`. Template literals for the label. Docs: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/reduce. Search: "javascript find max object two keys", "typescript readonly array".

Hint 3, plan. (1) If the list is empty, return null. (2) Start with the first element as the best so far. (3) For each other element: it is later if its season is higher, or if the season is equal and its episode is higher. (4) Return the best. (5) For the label, handle null first, then build the text.

Check. `npm test` passes: `latestEpisode` of [S1E10, S2E3, S2E1] → `{ seasonNumber: 2, episodeNumber: 3 }` (use `toEqual`). `latestEpisode([])` → `null`. `progressLabel({ seasonNumber: 2, episodeNumber: 5 })` → `"S2 E5"`. `progressLabel(null)` → `"Not started"`. `latestEpisode([S1E1])` → S1E1.

Common mistake. Using `toBe` to compare objects. `toBe` checks that it is the same object, `toEqual` compares the contents.

3.5 The poster badge

Task. Define the `Badge` type and write `progressBadge`, with tests.

What this means. Every poster gets one of three badges, and the rules belong in one tested place instead of inside a screen. A title that is Done gets a check mark, whatever its type. A movie that is not done gets nothing, because a movie has no episodes to count. A series in Watching gets its percentage. A series in Watchlist gets nothing, because you have not started it. The return value is a discriminated union, so the component in Milestone 7 can switch on `kind` and TypeScript knows when `value` exists. When it works, five tests pass.

Shape.

- File: `src/domain/progress.ts`.
- `type Badge = { kind: 'check' } | { kind: 'percent'; value: number } | { kind: 'none' }` (exported).
- `progressBadge(title: TrackedTitle): Badge`. Uses `progressPercent` for the value.

Hint 1, direction. Rules in order of priority: status Done first, then media type, then status again.

Hint 2, tools. Discriminated unions: https://www.typescriptlang.org/docs/handbook/2/narrowing.html#discriminated-unions. Search: "typescript discriminated union kind". In tests you need full `TrackedTitle` objects: write one helper in the test file that returns a default title and lets you override some fields with the spread syntax `{ ...defaults, ...overrides }`.

Hint 3, plan. (1) Export the `Badge` type. (2) If the status is done, return the check. (3) If it is a movie, return none. (4) If the status is watchlist, return none. (5) Otherwise return percent with `progressPercent(watchedCount, totalEpisodes)`. (6) In the test file, write a helper `makeTitle(overrides: Partial<TrackedTitle>): TrackedTitle`.

Check. `npm test` passes: tv, done → `{ kind: 'check' }`. movie, done → `{ kind: 'check' }`. movie, watching → `{ kind: 'none' }`. tv, watching, 6 of 24 → `{ kind: 'percent', value: 25 }`. tv, watchlist, 0 of 10 → `{ kind: 'none' }`.

Common mistake. Checking the media type before the status, so a finished movie never gets its check mark.

Stuck?

If you see	Look at
`Cannot find name 'describe'` in Neovim	The template ships Jest types. Restart the TS server (`:LspRestart`) after the first test file exists.
Test shows `Expected: {...} Received: {...}` with equal-looking objects	You used `toBe` where you need `toEqual`.
`Object is possibly 'undefined'` on `watched[0]`	That is `noUncheckedIndexedAccess` working. Check the length first or handle `undefined`.
`Type ... is missing the following properties`	A test object misses fields. Use the `makeTitle` helper.

Done when

- `src/domain/types.ts` holds all the types from steps 3.1 and 3.2.
- 15 tests pass for `progressPercent`, `latestEpisode`, `progressLabel` and `progressBadge`.
- `npm run check` passes.
- README: Milestone 3 ticked.
- Committed with a message that names the milestone (`Milestone 3: domain types and progress`).

Answer without looking

3.1. Why are anime not a separate `MediaType`?

3.2. Why does `progressBadge` return an object with `kind` instead of just a number or `null`?

3.3. What makes `progressPercent` a pure function, and why does that make it easy to test?

3.4. Why does `latestEpisode` take a `readonly` array?

3.5. Why are specials (season 0) left out of the episode count?

MILESTONE 4

Talking to TMDB

Goal. Fetch search results, series, seasons and movies from TMDB, and turn TMDB's responses into your own domain types, tested against real saved responses.

You will learn. REST and HTTP · Bearer tokens · async/await and fetch · `unknown` at trust boundaries · fixtures · mapping functions · keeping secrets out of git

Before you start.

- Create a free TMDB account, then open https://www.themoviedb.org/settings/api and request API access. You need the "API Read Access Token" (the long one).
- TMDB authentication: https://developer.themoviedb.org/docs/authentication-application
- TMDB image basics: https://developer.themoviedb.org/docs/image-basics
- API reference pages: https://developer.themoviedb.org/reference/search-multi, https://developer.themoviedb.org/reference/tv-series-details, https://developer.themoviedb.org/reference/tv-season-details, https://developer.themoviedb.org/reference/movie-details
- MDN, Using the Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- Cheat sheet D (HTTP and TMDB).

Briefing

The idea in plain words

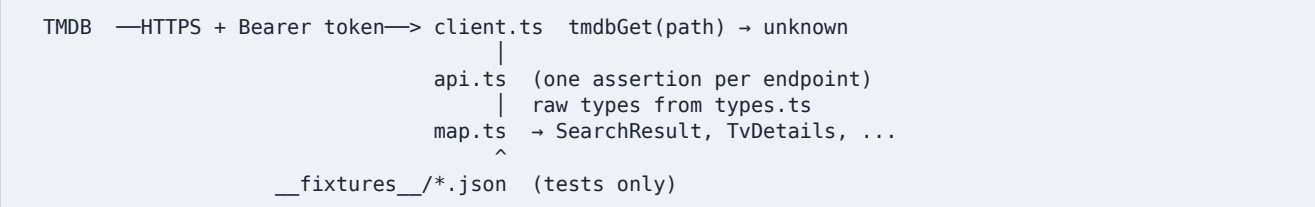
TMDB is a **REST API**: every piece of data has a web address (an endpoint), and you get it with an HTTP GET request. The answer is JSON text. TMDB needs to know who is asking, so every request carries your token in an `Authorization` header. If the token is wrong, TMDB answers with status 401 instead of data.

The responses use TMDB's own field names and include far more than you need. You will not pass them around the app. Instead, `src/tmdb/map.ts` turns each response into one of your domain types from Milestone 3. That function is pure, so you can test it without the internet: you save a few real responses as JSON files once (**fixtures**) and feed them to the mapping functions in tests. If TMDB ever changes something, only `map.ts` and its tests change.

The token is a password. It goes in `src/config/secrets.ts`, which git ignores. The repository only contains `secrets.example.ts` with a placeholder, so you remember which file to create on a new computer. Be aware that the token still ends up inside the app on your phone. That is fine for a personal app, but you should never publish this app as it is.

Data from the network has no guaranteed shape. That is why the low-level fetch function returns `unknown`, the type that allows nothing until you say what it is. In `api.ts` you state "this is a `TmdbTvDetails`" with a type assertion, at exactly one place per endpoint. That place is the **trust boundary**: everything after it is typed.

Where it fits



New words

Term	Meaning
REST API	A web API where each resource has a URL and you read it with HTTP GET.
Endpoint	One URL path of an API, like `/tv/{series_id}`.
Query parameter	A value after `?` in a URL, like `?query=matrix`.
HTTP status code	A number in the response saying how it went: 200 OK, 401 not allowed, 404 not found, 429 too many requests.
Bearer token	A secret string sent as `Authorization: Bearer <token>` to prove who you are.
`async` / `await`	Syntax for code that waits for a result (a Promise) without blocking the app.
`unknown`	A type that holds anything but lets you do nothing with it until you narrow or assert it.
Type assertion	`value as Type`: telling TypeScript what a value is. TypeScript trusts you, so use it only at a boundary.
Fixture	Saved example data used as test input.
Mapping function	A function that turns one data shape into another.

Steps

4.1 Store the token outside git

Task. Create `src/config/secrets.ts` with your token, make git ignore it, and commit `src/config/secrets.example.ts` with a placeholder.

What this means. Other files will import the token from `src/config/secrets.ts`. Git must never see that file, so you add it to `.gitignore` before you create it. The example file has the same export with a fake value, so anyone (including you on a new computer) knows what to create. When it works, git ignores the real file and tracks the example file.

Shape.

- `src/config/secrets.ts`: `export const TMDB_READ_TOKEN: string` = your real token.
- `src/config/secrets.example.ts`: the same export, value `'paste-your-tmdb-read-access-token-here'`.
- `.gitignore`: one new line for `src/config/secrets.ts`.

Hint 1, direction. Ignore first, create second. Git only ignores files it is not already tracking.

Hint 2, tools. `.gitignore` patterns, `git check-ignore`, `git status`. Docs: https://git-scm.com/docs/gitignore. Search: "git check-ignore", "gitignore single file path".

Hint 3, plan. (1) Add the path to `.gitignore`. (2) Create the example file. (3) Copy it to `secrets.ts` and paste your real token. (4) Run `git status`.

Check. `git check-ignore src/config/secrets.ts` prints the path. `git status` lists `secrets.example.ts` and `.gitignore`, but not `secrets.ts`.

Common mistake. Using the short "API Key" instead of the long "API Read Access Token". The Bearer header needs the long one.

4.2 Save four real responses as fixtures

Task. With curl, save four TMDB responses as JSON files in `src/tmdb/__fixtures__/`: a search for "matrix", Breaking Bad's series details, Breaking Bad season 1, and the movie The Matrix.

What this means. These files are the test input for the next steps. Saving real responses means your tests check against what TMDB actually sends, not what you think it sends. Breaking Bad (TMDB id 1396) is a good series fixture because it has finished: its episode counts will not change. The Matrix has TMDB id 603. You put your token in a shell variable for this terminal only, so it never lands in your shell history file as plain text in a command. When it works, the four files exist and contain JSON.

Shape.

- Folder: `src/tmdb/__fixtures__/`.
- `search-matrix.json` ← `GET /search/multi?query=matrix`
- `tv-1396.json` ← `GET /tv/1396`
- `tv-1396-season-1.json` ← `GET /tv/1396/season/1`
- `movie-603.json` ← `GET /movie/603`
- Base URL `https://api.themoviedb.org/3`, header `Authorization: Bearer <token>`.

Hint 1, direction. One curl command per file: the URL, the header, and an output file.

Hint 2, tools. curl options `-sS` (quiet but show errors), `-H` (header), `-o` (output file). Example shape: `curl -sS -H "Authorization: Bearer $TMDB_TOKEN" "<url>" -o <file>`. Docs: https://curl.se/docs/manpage.html. Search: "curl bearer token header", "curl -o save to file".

Hint 3, plan. (1) Run `export TMDB_TOKEN='<token>'` in this terminal. (2) Create the folder. (3) Run curl once per endpoint with the matching output file. (4) Open each file in Neovim and look at the field names you will need in step 4.3.

Check. `grep -c '"name":"Breaking Bad"' src/tmdb/__fixtures__/tv-1396.json` prints 1 or more. `grep -o '"runtime":136' src/tmdb/__fixtures__/movie-603.json` prints `"runtime":136`. None of the four files contains `status_code` (TMDB's error format).

Common mistake. Single quotes around the URL work, but single quotes around the header stop `$TMDB_TOKEN` from being replaced, so TMDB gets the literal text `$TMDB_TOKEN`.

4.3 Describe TMDB's raw responses

Task. Create `src/tmdb/types.ts` with types for the parts of the four responses that you will use.

What this means. Before you can map a response, TypeScript needs to know its shape. You only describe the fields you use, with TMDB's exact names. Look in your fixtures to see which fields can be `null` or empty. The search response mixes three kinds of result, told apart by `media_type`, and `'person'` results will be thrown away later. That is a discriminated union again, this time on TMDB's data. When it works, the type check passes.

Shape.

- `TmdbSearchResponse`: `page`, `results: TmdbSearchItem[]`, `total_pages`, `total_results`.
- `TmdbSearchItem` = `TmdbSearchMovie | TmdbSearchTv | TmdbSearchPerson`, told apart by `media_type: 'movie' | 'tv' | 'person'`.
- Movie item: `id`, `title`, `release_date?: string`, `poster_path: string | null`. TV item: `id`, `name`, `first_air_date?: string`, `poster_path: string | null`. Person item: `id`, `name`.
- `TmdbTvDetails`: `id`, `name`, `overview`, `poster_path`, `seasons` (each `season_number`, `name`, `episode_count`), `next_episode_to_air` (`season_number`, `episode_number`, `air_date`) or `null`.
- `TmdbSeasonDetails`: `season_number`, `episodes` (each `season_number`, `episode_number`, `name`, `air_date: string | null`).

- `TmdbMovieDetails`: `id`, `title`, `overview`, `poster_path`, `runtime: number | null`, `release_date: string`.

Hint 1, direction. Read the fixtures and copy the structure into types, keeping only what you need.

Hint 2, tools. Optional property (`?`) for fields that are sometimes missing, `| null` for fields that are sometimes null. Search: "typescript type json response", "typescript discriminated union string field".

Hint 3, plan. (1) Open `search-matrix.json` and write the three item types and their union. (2) Write the response type. (3) Do the same for the TV details, using a separate small type for one season and one for the next episode. (4) Season details. (5) Movie details.

Check. `npm run typecheck` passes.

Common mistake. Writing `media_type: string`. Then TypeScript cannot tell the three result kinds apart.

4.4 Map search results and movies

Task. In `src/tmdb/map.ts`, write `mapSearchResults` and `mapMovieDetails`, with tests that use the fixtures.

What this means. `mapSearchResults` turns TMDB's mixed list into `SearchResult[]`. It drops people, takes the name from `title` for movies and `name` for series, and takes the year from the first four characters of the date. TMDB sometimes sends an empty string or no date at all, and both must become `null`. `mapMovieDetails` renames the fields of a movie. In the tests you import the JSON fixture and pass it through the type from step 4.3. When it works, the search fixture produces only movies and series, and The Matrix comes out with the right year and runtime.

Shape.

- File: `src/tmdb/map.ts`. Tests: `src/tmdb/map.test.ts`.
- `mapSearchResults(raw: TmdbSearchResponse): SearchResult[]`. People are removed. Empty or missing date → `year: null`.
- `mapMovieDetails(raw: TmdbMovieDetails): MovieDetails`. Empty `release_date` → `null`.

Hint 1, direction. Filter, then map. Narrow on `media_type` inside the loop so TypeScript knows which fields exist.

Hint 2, tools. `Array.prototype.filter`, `Array.prototype.map` (or one loop), `String.prototype.slice`. Importing JSON in tests: `import raw from './__fixtures__/movie-603.json'`, then `raw as TmdbMovieDetails`. JSON imports already work, because the React Native TypeScript config turns on `resolveJsonModule`. Search: "typescript import json resolveJsonModule", "typescript narrowing switch union".

Hint 3, plan. (1) Write a small helper that turns a date string into a year or `null`. (2) Loop over results and skip people. (3) For movies and series, build a `SearchResult` with the right name field. (4) Write `mapMovieDetails` field by field. (5) In the test file, import both fixtures and write the cases below.

Check. `npm test` passes: the mapped `search-matrix.json` contains no item whose `mediaType` is anything other than `'movie'` or `'tv'`. It contains an item with `tmdbId: 603`, `mediaType: 'movie'`, `name: 'The Matrix'`, `year: '1999'`. The mapped `movie-603.json` has `name: 'The Matrix'` and `runtimeMinutes: 136`.

Common mistake. Expecting every `.filter()` to narrow. Since TypeScript 5.5 a simple callback such as `item => item.media_type !== 'person'` narrows the result on its own, but as soon as the callback does more (a block body, a second condition on another field) TypeScript may keep the full union. If the narrowed type does not appear, write an explicit type guard or narrow inside a loop.

4.5 Map series and seasons

Task. Add `mapTvDetails` and `mapSeason` to `src/tmdb/map.ts`, with tests.

What this means. `mapTvDetails` renames the fields and does two pieces of real work. It removes season 0 (specials) from the season list, and it calculates `totalEpisodes` as the sum of `episode_count` over the remaining seasons. Doing it yourself means you control the rule from Milestone 3 instead of trusting another field. It also turns `next_episode_to_air` into your `NextEpisode` or `null`. `mapSeason` turns a season response into a list of `Episode`. When it works, Breaking Bad comes out with 5 seasons and 62 episodes.

Shape.

- `mapTvDetails(raw: TmdbTvDetails): TvDetails`. Seasons with `season_number` 0 are removed.
- `mapSeason(raw: TmdbSeasonDetails): Episode[]`. Empty `air_date` → `null`.

Hint 1, direction. Filter the seasons first, then sum. Everything else is renaming.

Hint 2, tools. `filter` then `reduce` (or a loop) for the sum. Search: "javascript sum array of objects reduce".

Hint 3, plan. (1) Filter out season 0. (2) Map each remaining season to `SeasonSummary`. (3) Sum their episode counts. (4) Map the next episode if present. (5) Build `TvDetails`. (6) For `mapSeason`, map each episode. (7) Write the tests.

Check. `npm test` passes: mapped `tv-1396.json` has `name: 'Breaking Bad'`, `seasons` of length 5, `seasons[0]` with `seasonNumber: 1` and `episodeCount: 7`, `totalEpisodes: 62`, and `nextEpisode: null`. Mapped `tv-1396-season-1.json` has length 7, and its first element has `episodeNumber: 1` and `name: 'Pilot'`.

Common mistake. Using `seasons[0]` without a check in the main code. With `noUncheckedIndexedAccess` that is an error, and a series with no seasons would crash.

4.6 Build poster URLs

Task. Write `posterUrl` in `src/tmdb/images.ts`, with tests.

What this means. TMDB only gives you the end of an image path, like `/abc.jpg`. A full URL also needs the image server's base address and a size. Smaller sizes load faster, so the grid will use `w342` and the search list `w185`. A title without a poster has `posterPath: null`, and then there is no URL either. When it works, three tests pass.

Shape.

- File: `src/tmdb/images.ts`. Tests: `src/tmdb/images.test.ts`.
- `type PosterSize = 'w185' | 'w342' | 'w500'` (exported).
- `TMDB_IMAGE_BASE = 'https://image.tmdb.org/t/p'` (exported constant).
- `posterUrl(posterPath: string | null, size: PosterSize): string | null`

Hint 1, direction. Null check, then join three parts.

Hint 2, tools. Image URL format from https://developer.themoviedb.org/docs/image-basics. Template literals. Search: "tmdb image url poster_path".

Hint 3, plan. (1) If the path is null, return null. (2) Return base + `/` + size + path (the path already starts with `/`).

Check. `npm test` passes: `posterUrl('/abc.jpg', 'w342')` → `'https://image.tmdb.org/t/p/w342/abc.jpg'`. `posterUrl('/abc.jpg', 'w185')` → `'https://image.tmdb.org/t/p/w185/abc.jpg'`. `posterUrl(null, 'w342')` → `null`.

Common mistake. Adding a second `/` before the path, which gives `w342//abc.jpg`.

4.7 The HTTP client and the API functions

Task. Write `tmdbGet` and `TmdbError` in `src/tmdb/client.ts`, and the four API functions in `src/tmdb/api.ts`.

What this means. `tmdbGet` is the only function in the app that calls `fetch`. It builds the URL with query parameters, sends the token, throws a `TmdbError` with the status code if the answer is not OK, and returns the parsed JSON as `unknown`. The API functions call it, assert the raw type and map the result. An empty search returns an empty list without calling TMDB at all. A network failure (no internet) makes `fetch` throw, and you let that error pass up to the screen. When it works, a temporary log in the app prints the number of results.

Shape.

- `src/tmdb/client.ts`: `TMDB_BASE_URL = 'https://api.themoviedb.org/3'`. `class TmdbError extends Error` with `readonly status: number`. `tmdbGet(path: string, params?: Record<string, string>): Promise<unknown>`.
- `src/tmdb/api.ts`: `searchTitles(query: string): Promise<SearchResult[]>` (trimmed empty query → `[]`, no request), `getTvDetails(tmdbId: number): Promise<TvDetails>`, `getSeason(tmdbId: number, seasonNumber: number): Promise<Episode[]>`, `getMovieDetails(tmdbId: number): Promise<MovieDetails>`.
- No tests for these two files: they need the network. You check them by running the app.

Hint 1, direction. One generic GET helper, then thin wrappers: request, assert, map.

Hint 2, tools. `fetch`, `response.ok`, `response.status`, `response.json()`, `URLSearchParams`, `class ... extends Error`. Docs: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch, https://developer.mozilla.org/en-US/docs/Web/API/URLSearchParams. Search: "fetch authorization header bearer", "typescript custom error class".

Hint 3, plan. (1) Write `TmdbError` with a constructor that takes the status and sets a message. (2) In `tmdbGet`, build the URL from base, path and params. (3) Fetch with the Authorization header. (4) If not ok, throw `TmdbError`. (5) Return `await response.json()`. (6) In `api.ts`, write each function: call `tmdbGet` with the endpoint path, assert the raw type, return the mapped value. (7) Temporarily, in `App.tsx`, add a `useEffect` that calls `searchTitles('matrix')` and logs the length of the result (see Cheat sheet F for `useEffect`).

Check. Since React Native 0.77, `console.log` no longer shows in the Metro terminal. Press `j` in Metro to open React Native DevTools (or run `adb logcat ReactNativeJS:V *:S`): after the app loads, the console shows a number between 1 and 20. Temporarily change one character of your token and reload: the log shows a `TmdbError` with status 401. Restore the token, remove the temporary `useEffect`, and `npm run check` passes.

Common mistake. Forgetting `await` before `response.json()`, so the function returns a Promise of a Promise and the mapping receives the wrong thing.

Stuck?

If you see	Look at
Status 401	Long token vs short key (step 4.1), or `Bearer ` missing in the header.
`Cannot find module './__fixtures__/x.json'`	File name and relative path from the test file.
`Network request failed` on the phone	Phone internet, or `https` missing in the base URL.
A test gets `undefined` for a field	Field name spelling in `types.ts` against the fixture.
`Conversion of type ... may be a mistake`	You asserted directly from a JSON import with a different shape. Check your raw type against the fixture.

Done when

- The token is in `src/config/secrets.ts` and git ignores it.
- Four fixtures exist, and the mapping tests pass on them.
- `posterUrl` is tested.
- A temporary search from the app printed results, then the temporary code was removed.
- `npm run check` passes and the milestone is ticked in the README.
- Committed with a message that names the milestone (`Milestone 4: TMDB client and mapping`).

Answer without looking

4.1. Why does `tmdbGet` return `unknown` instead of `any`?

4.2. Why test `map.ts` with saved fixtures instead of calling TMDB in the tests?

4.3. Why calculate `totalEpisodes` yourself instead of using a field from TMDB?

4.4. The token is git-ignored. Is it therefore secret on your phone? Explain.

4.5. What should `searchTitles(' ')` do, and why?

MILESTONE 5

Designing the database

Goal. Store your titles and watched episodes in SQLite on the phone, with tables you design and queries you test on your computer.

You will learn. tables and columns · primary keys · UNIQUE · CHECK · foreign keys and ON DELETE CASCADE · parameterized queries · JOIN and GROUP BY · migrations with user_version · interfaces for testability · type guards

Before you start.

- op-sqlite installation and API: https://op-engineering.github.io/op-sqlite/docs/installation and https://op-engineering.github.io/op-sqlite/docs/api
- SQLite: CREATE TABLE https://www.sqlite.org/lang_createtable.html, foreign keys https://www.sqlite.org/foreignkeys.html, UPSERT https://www.sqlite.org/lang_upsert.html, PRAGMA user_version https://www.sqlite.org/pragma.html#pragma_user_version
- better-sqlite3 API: https://github.com/WiseLibs/better-sqlite3/blob/master/docs/api.md
- TypeScript interfaces: https://www.typescriptlang.org/docs/handbook/2/objects.html, type predicates: https://www.typescriptlang.org/docs/handbook/2/narrowing.html#using-type-predicates
- Cheat sheet E (SQL and SQLite).

Briefing

The idea in plain words

SQLite is a full SQL database stored in one file. On the phone, op-sqlite gives you access to it. You need two tables. `titles` has one row per movie or series you added. `watched_episodes` has one row per episode you marked as watched, pointing at its title. The progress percentage is never stored: it is counted from `watched_episodes` when you read the list, so it can never be out of date.

op-sqlite is a native library and does not run in Jest on your computer. To test your SQL anyway, your query functions do not use op-sqlite directly. They take a parameter of your own type `Db`, an **interface** with one method: `execute(sql, params)`. On the phone, `appDb.ts` wraps op-sqlite into a `Db`. In tests, `testDb.ts` wraps better-sqlite3, a SQLite library for Node, around an in-memory database. Both run real SQLite, so a passing test means the query really works. This idea is called **dependency injection**: the function receives what it depends on instead of creating it.

Values in your SQL always go through `?` placeholders and the `params` array, never by building the SQL string yourself. The library then sends values separately from the SQL text, so a title like `Grey's Anatomy` cannot break the query.

Databases outlive app versions. If you change a table in a later version, the phone still has the old one. A **migration** is a numbered change to the schema. SQLite has a free integer slot called `user_version`: you read it at start-up, run every migration with a higher number, and store the new number. This milestone has migration 1. Your future self adds migration 2.

Rows come back as plain objects whose values could be any SQL type. When you turn a row into a `TrackedTitle`, you check each value's type at runtime. A **type guard** is a function returning `value is Status` that tells TypeScript "if this returned true, the value is a `Status`".

Where it fits



New words

Term	Meaning
Primary key (PK)	The column or columns that identify one row. No two rows may share it.
Foreign key (FK)	A column that must match the primary key of a row in another table.
ON DELETE CASCADE	When the parent row is deleted, its child rows are deleted too.
UNIQUE	A constraint: no two rows may have the same value (or combination of values) here.
CHECK	A constraint: the value must satisfy a condition, for example one of three statuses.
Parameterized query	SQL with `?` placeholders. The values are passed separately.
LEFT JOIN	Combine rows from two tables, keeping left rows even when nothing matches on the right.
GROUP BY	Collapse rows with the same value into one row, so you can count them.
Migration	A numbered change to the database schema, run once per device.
Interface	A named object type, often describing what something can do (its methods).
Dependency injection	Passing a function the things it depends on, instead of letting it create them.
Type guard	A function with return type `value is T` that narrows a value's type when it returns true.

Steps

5.1 Install the two SQLite libraries

Task. Install `@op-engineering/op-sqlite` for the app and `better-sqlite3` with its types for tests, then rebuild the app.

```
npm install @op-engineering/op-sqlite
npm install --save-dev better-sqlite3 @types/better-sqlite3
npm run android
```

What this means. op-sqlite contains native C++ code, so adding it changes the native half of the app. Fast Refresh is not enough: you need a new build. better-sqlite3 is only a development dependency. It runs on your computer in Jest and never goes into the app. When it works, the app builds and still shows your screen.

Shape.

- `package.json`: op-sqlite under `dependencies`, better-sqlite3 and its types under `devDependencies`.

Hint 1, direction. The commands above are the whole step. Notice which list each package lands in.

Hint 2, tools. Docs: https://op-engineering.github.io/op-sqlite/docs/installation. Search: "npm save-dev vs dependencies".

Hint 3, plan. (1) Run the three commands with Metro running in another terminal. (2) Open `package.json` and find the three new entries.

Check. `npm run android` ends with `BUILD SUCCESSFUL` and the phone shows "Media Tracker". `grep better-sqlite3 package.json` shows it inside `devDependencies`.

Common mistake. Forgetting to rebuild. The app then crashes as soon as code imports op-sqlite.

5.2 The Db interface and a test database

Task. Define the `Db` interface in `src/db/database.ts`, and write `createTestDb` in `src/db/testDb.ts` on top of better-sqlite3.

What this means. This is the dependency injection idea from the briefing. `database.ts` only contains types, so any file can import it without pulling in a native library. `createTestDb` opens an in-memory SQLite database (it disappears when the test ends), turns on foreign keys, and returns an object that fits `Db`. better-sqlite3 has one method for statements that return rows (`all`) and another for statements that change data (`run`). Your `execute` picks the right one by asking the prepared statement whether it returns rows. When it works, a test runs real SQL through your interface.

Shape.

- `src/db/database.ts`: `type SqlValue = string | number | null`, `type Row = Record<string, SqlValue>`, `type QueryResult = { rows: Row[]; rowsAffected: number; insertId: number | undefined }`, `interface Db { execute(sql: string, params?: SqlValue[]): Promise<QueryResult> }`.
- `src/db/testDb.ts`: `createTestDb(): Db`. Foreign keys on. Only tests import this file.
- Test: `src/db/testDb.test.ts`.

Hint 1, direction. Wrap a synchronous library behind an asynchronous interface. An `async` function always returns a Promise, even if it never waits.

Hint 2, tools. better-sqlite3: `new Database(':memory:')`, `db.prepare(sql)`, `statement.reader`, `statement.all(...params)`, `statement.run(...params)` → `{ changes, lastInsertRowid }`, `db.pragma(...)`. Docs: https://github.com/WiseLibs/better-sqlite3/blob/master/docs/api.md. Search: "better-sqlite3 reader property", "better-sqlite3 in memory database".

Hint 3, plan. (1) Write the four types in `database.ts`. (2) In `createTestDb`, open an in-memory database and turn on foreign keys. (3) Return an object with an async `execute`. (4) Inside: prepare the SQL. If it returns rows, get them all and report 0 rows affected. Otherwise run it and turn `changes` and `lastInsertRowid` into numbers. (5) The `rows` from better-sqlite3 are `unknown[]`: assert them to `Row[]` here, at this one boundary.

Check. `npm test` passes a test that runs `SELECT 1 + 1 AS two` through `createTestDb()` and expects `rows[0]?.two` to be `2`, and a second test that runs `CREATE TABLE t (x INTEGER)` then `INSERT INTO t (x) VALUES (?)` with params `[5]` and expects `rowsAffected` to be `1`.

Common mistake. Calling `all` on a CREATE or INSERT. better-sqlite3 throws "This statement does not return data. Use run() instead". That is why `execute` must check `reader` first.

5.3 The schema and the first migration

Task. Write `migrate` in `src/db/schema.ts`. It creates both tables as migration 1 and records the version in `user_version`.

What this means. `migrate` runs every time the app starts. It reads `PRAGMA user_version`. A brand-new database has 0. If the version is below 1, it creates the two tables and sets `user_version` to 1. On the next start it reads 1 and does nothing. Running it twice must therefore be safe. The constraints from the briefing belong in the table definitions, so the database itself refuses bad data, even if your code has a bug. When it works, a migrated test database has exactly two tables and version 1.

Shape.

- File: `src/db/schema.ts`. Export `migrate(db: Db): Promise<void>`. Tests: `src/db/schema.test.ts`.
- `titles`: `id INTEGER PRIMARY KEY`, `tmdb_id INTEGER NOT NULL`, `media_type TEXT NOT NULL` (CHECK: 'movie' or 'tv'), `name TEXT NOT NULL`, `poster_path TEXT`, `total_episodes INTEGER NOT NULL DEFAULT 0`, `status TEXT NOT NULL` (CHECK: the three statuses), `added_at INTEGER NOT NULL`, `last_watched_at INTEGER`, and `UNIQUE (tmdb_id, media_type)`.
- `watched_episodes`: `title_id INTEGER NOT NULL` referencing `titles(id)` with `ON DELETE CASCADE`, `season_number INTEGER NOT NULL`, `episode_number INTEGER NOT NULL`, `watched_at INTEGER NOT NULL`, primary key `(title_id, season_number, episode_number)`.
- One SQL statement per `execute` call (op-sqlite runs only the first statement of a string).

Hint 1, direction. Read a number, compare, run several statements, write the number.

Hint 2, tools. `PRAGMA user_version` (read) and `PRAGMA user_version = 1` (write). `CREATE TABLE`, `CHECK (status IN (...))`, `REFERENCES ... ON DELETE CASCADE`. Docs: https://www.sqlite.org/lang_createtable.html, https://www.sqlite.org/pragma.html#pragma_user_version. Search: "sqlite user_version migrations", "sqlite composite primary key".

Hint 3, plan. (1) Execute `PRAGMA user_version` and read the `user_version` field of the first row. Treat a missing row as 0. (2) If it is below 1: create `titles`, create `watched_episodes`, then set the version to 1. (3) In the test, migrate a fresh test database twice, then query the version and the table names.

Check. `npm test` passes: after calling `migrate` twice on `createTestDb()`, `PRAGMA user_version` returns `user_version: 1`, and `SELECT name FROM sqlite_master WHERE type = 'table' ORDER BY name` returns exactly the names `titles` and `watched_episodes`. A third test: inserting a title with status `'paused'` rejects (use `await expect(...).rejects.toThrow()`).

Common mistake. Writing `PRAGMA user_version = ?` with a parameter. SQLite's PRAGMA syntax only allows a literal number or name as the value, so write the number directly.

5.4 Add and read one title

Task. Add `NewTitle` to `src/domain/types.ts`, and write `addTitle` and `getTitle` in `src/db/titles.ts`, with tests.

What this means. `addTitle` saves a title you chose to track. Adding the same TMDB title twice must not create a second row: the UNIQUE constraint would reject it, so you tell SQLite to do nothing on that conflict, and then return the id of the existing row. The current time is a parameter instead of calling `Date.now()` inside, so tests can use fixed numbers. `getTitle` finds a title by TMDB id and type, and turns the row into a `TrackedTitle`. That conversion checks every value's type at runtime and throws if the database has something unexpected. When it works, adding Breaking Bad twice gives the same id and one row.

Shape.

- `src/domain/types.ts`: `NewTitle` = the fields `tmdbId`, `mediaType`, `name`, `posterPath`, `totalEpisodes`, `status` of `TrackedTitle`.
- `addTitle(db: Db, input: NewTitle, now: number): Promise<number>` returns the row id (new or existing).

- `getTitle(db: Db, mediaType: MediaType, tmdbId: number): Promise<TrackedTitle | null>` (`watchedCount` comes from counting `watched_episodes`).
- Inside `titles.ts` (not exported): `rowToTitle(row: Row): TrackedTitle`, `isStatus(value: unknown): value is Status`, `isMediaType(value: unknown): value is MediaType`. A wrong value → throw an `Error` naming the column.
- Tests: `src/db/titles.test.ts`. Each test starts from a new migrated test database.

Hint 1, direction. An upsert that ignores conflicts, followed by a lookup. For rows, check types with `typeof` and your type guards.

Hint 2, tools. `Pick<TrackedTitle, ...>`. SQL: `INSERT ... ON CONFLICT (tmdb_id, media_type) DO NOTHING`, `SELECT ... WHERE ... = ? AND ... = ?`, a subquery `(SELECT COUNT(*) FROM watched_episodes WHERE title_id = titles.id) AS watched_count`. Jest: `beforeEach`. Docs: https://www.sqlite.org/lang_upsert.html, https://www.typescriptlang.org/docs/handbook/utility-types.html#picktype-keys. Search: "sqlite insert on conflict do nothing", "typescript type predicate value is".

Hint 3, plan. (1) Write `NewTitle` with `Pick`. (2) Write the two type guards. (3) Write `rowToTitle`, reading each column and checking its type. Columns are snake_case, fields are camelCase. (4) `addTitle`: insert with conflict-do-nothing, then select the id by tmdb id and type. (5) `getTitle`: select the row with the counted subquery and convert it, or return null. (6) Tests with `beforeEach` creating and migrating a fresh database.

Check. `npm test` passes: `addTitle` for Breaking Bad (tmdbId 1396, tv, totalEpisodes 62, watching, now 1000) returns `1`. Calling it again returns `1`, and `SELECT COUNT(*) AS n FROM titles` gives `n: 1`. `getTitle(db, 'tv', 1396)` has `name: 'Breaking Bad'`, `watchedCount: 0`, `addedAt: 1000`, `lastWatchedAt: null`. `getTitle(db, 'movie', 1396)` → `null`.

Common mistake. Trusting `insertId` after a conflict. When nothing was inserted, it does not point at the existing row.

5.5 List titles by status and change status

Task. Write `listTitlesByStatus` and `setStatus` in `src/db/titles.ts`, with tests.

What this means. This is the query behind the main screen grid. It returns all titles with one status, each with its count of watched episodes, sorted so the one you touched most recently comes first. "Touched" means last watched, or added if you have never watched it. Counting uses a LEFT JOIN, so titles with zero watched episodes still appear. `setStatus` moves a title between Watching, Done and Watchlist. When it works, titles come back in the right order and move between lists.

Shape.

- `listTitlesByStatus(db: Db, status: Status): Promise<TrackedTitle[]>`. Order: newest of (`last_watched_at`, or else `added_at`) first.
- `setStatus(db: Db, id: number, status: Status): Promise<void>`. An unknown id changes nothing and does not throw.

Hint 1, direction. LEFT JOIN plus GROUP BY plus COUNT, and an ORDER BY that falls back from one column to another.

Hint 2, tools. SQL: `LEFT JOIN watched_episodes ON ...`, `COUNT(watched_episodes.title_id)`, `GROUP BY titles.id`, `COALESCE(a, b)`, `ORDER BY ... DESC`, `UPDATE ... SET ... WHERE ...`. Reuse `rowToTitle`. Docs: https://www.sqlite.org/lang_select.html, https://www.sqlite.org/lang_corefunc.html#coalesce. Search: "sqlite left join count group by", "sqlite coalesce order by".

Hint 3, plan. (1) Select all title columns plus the count, from titles left-joined to watched episodes. (2) Filter by status. (3) Group by title id. (4) Order by the coalesced time, newest first. (5) Map rows with `rowToTitle`. (6) `setStatus` is one UPDATE.

Check. `npm test` passes: add A (tmdbId 1, tv, watching, now 100), B (tmdbId 2, tv, watching, now 200), C (tmdbId 3, movie, watchlist, now 300). `listTitlesByStatus(db, 'watching')` gives names in order [B, A]. After `setStatus` on A to `'done'`, Watching gives [B] and Done gives [A]. Watchlist gives [C].

Common mistake. `COUNT(*)` instead of counting a column from the right table. With LEFT JOIN, `COUNT(*)` gives 1 for a title with no episodes.

5.6 Mark and unmark episodes

Task. Write `markEpisode`, `unmarkEpisode` and `listWatched` in `src/db/episodes.ts`, with tests.

What this means. Marking an episode inserts one row, and also updates the title's `last_watched_at`, so it moves to the front of the grid. Marking the same episode twice must not fail or count double: the primary key makes the second insert a conflict, and you ignore it. Unmarking deletes the row. `listWatched` returns every watched episode of one title, which the details page will need. When it works, counts match what you marked, and the grid order changes after marking.

Shape.

- File: `src/db/episodes.ts`. Tests: `src/db/episodes.test.ts`.
- `markEpisode(db: Db, titleId: number, ref: EpisodeRef, now: number): Promise<void>`
- `unmarkEpisode(db: Db, titleId: number, ref: EpisodeRef): Promise<void>`
- `listWatched(db: Db, titleId: number): Promise<EpisodeRef[]>` ordered by season, then episode.

Hint 1, direction. Two statements for marking (insert-or-ignore, then update the title). One statement each for the others.

Hint 2, tools. `INSERT OR IGNORE INTO ...`, `DELETE FROM ... WHERE ...`, `ORDER BY a, b`. Check numbers with `typeof` when reading rows. Search: "sqlite insert or ignore", "sqlite delete where multiple columns".

Hint 3, plan. (1) `markEpisode`: insert or ignore the row, then set `last_watched_at` on the title. (2) `unmarkEpisode`: delete by the three key columns. (3) `listWatched`: select season and episode numbers, ordered, mapped to `EpisodeRef`. (4) Tests on a migrated test database with one title added first.

Check. `npm test` passes: add Breaking Bad (now 100). Mark S1E1 and S1E2 (now 200): `listWatched` has length 2. Mark S1E2 again: still 2. Unmark S1E1: `listWatched` equals `[{ seasonNumber: 1, episodeNumber: 2 }]`. `getTitle` now has `watchedCount: 1` and `lastWatchedAt: 200`. Extra ordering test: with A (now 100) and B (now 200) watching, marking an episode on A at now 300 makes `listTitlesByStatus(db, 'watching')` return [A, B].

Common mistake. Forgetting to update `last_watched_at`, so marking an episode never moves a title to the front.

5.7 The real database on the phone

Task. Write `getAppDb` in `src/db/appDb.ts`. It opens op-sqlite once, wraps it as a `Db`, turns on foreign keys, migrates, and caches the result.

What this means. Screens will call `getAppDb()` whenever they need the database. Opening and migrating must happen only once per app start, even if two screens ask at the same time. You do that by storing the Promise in a module-level variable the first time and returning the same Promise afterwards. The wrapper converts op-sqlite's result into your `QueryResult`. This is the only file that imports op-sqlite, which is why no test imports it. When it works, a temporary query in the app logs the number of titles.

Shape.

- File: `src/db/appDb.ts`. Export `getAppDb(): Promise<Db>`.
- Database file name: `mediatracker.sqlite`.

- Foreign keys must be turned on with `PRAGMA foreign_keys = ON` for this connection before migrating.
- op-sqlite rows can contain value types your `SqlValue` does not list. Your schema only stores text, integers and null, so asserting the rows to `Row[]` here is acceptable. Write a one-line comment saying why.

Hint 1, direction. Cache the Promise, not the result. Then a second caller waits for the same work instead of starting it again.

Hint 2, tools. op-sqlite `open({ name })` and `db.execute(sql, params)` → `{ rows, rowsAffected, insertId }`. Docs: https://op-engineering.github.io/op-sqlite/docs/api. Search: "cache promise singleton typescript", "lazy initialization promise".

Hint 3, plan. (1) Declare a module variable `Promise<Db> | null`. (2) Write a private async function that opens op-sqlite, builds the `Db` wrapper, runs the foreign keys pragma and `migrate`, and returns the wrapper. (3) `getAppDb` returns the cached Promise, creating it on the first call. (4) Temporarily, in `App.tsx`, add a `useEffect` that gets the database, runs `SELECT COUNT(*) AS n FROM titles` and logs the first row.

Check. Metro prints `{"n": 0}` (or `{ n: 0 }`) when the app loads. Reload the app: it prints the same, with no error about tables already existing. Remove the temporary code, and `npm run check` passes.

Common mistake. Caching the database only after `await` finishes. Two screens starting at the same time then both open and migrate it.

Stuck?

If you see	Look at
`FOREIGN KEY constraint failed`	You inserted an episode for a title id that does not exist. Check the test's setup.
`UNIQUE constraint failed: titles.tmdb_id, titles.media_type`	Your `ON CONFLICT` clause in `addTitle`.
`CHECK constraint failed`	A status or media type that is not in the allowed list.
A test passes alone but fails together with others	Tests share a database. Create a new one in `beforeEach`.
`table titles already exists`	`migrate` does not read or set `user_version` correctly.
App crashes on start after 5.1	You did not rebuild with `npm run android`.
`rowToTitle` throws about a column	Column alias spelling in the SELECT (`watched_count`).

Done when

- ■ Schema, title and episode queries are tested against a real in-memory SQLite.
- ■ The app opens its own database on the phone and migrates it.
- ■ No file except `appDb.ts` imports op-sqlite, and no file except tests imports `testDb.ts`.
- ■ `npm run check` passes and the milestone is ticked in the README.
- ■ Committed with a message that names the milestone (`Milestone 5: SQLite schema and queries`).

Answer without looking

5.1. Why do the query functions take a `Db` parameter instead of importing op-sqlite?

5.2. Why is the progress percentage not stored in the `titles` table?

5.3. What does `user_version` protect you from when you release a new version with a changed table?

5.4. Why must values go through `?` placeholders instead of being put into the SQL string?

5.5. Why does `getAppDb` cache a Promise and not the finished `Db`?

MILESTONE 6

Screens and navigation

Goal. Set up a typed stack of three screens (Home, Search, Details) so you can move between them, pass parameters, and go back with the Android back button.

You will learn. components and props · native stack navigator · route parameters · typed screen props · native configuration after installing a library

Before you start.

- React Navigation, Getting started: https://reactnavigation.org/docs/getting-started
- Hello React Navigation (choose "Dynamic"): https://reactnavigation.org/docs/hello-react-navigation
- Passing parameters: https://reactnavigation.org/docs/params
- Type checking with TypeScript: https://reactnavigation.org/docs/typescript
- Cheat sheets F (React Native) and G (React Navigation).

Briefing

The idea in plain words

A navigator keeps a **stack** of screens. Opening a screen pushes it on top, and the back button pops it off. The Home screen is always at the bottom. Search and Details open on top of it, and Details can also open on top of Search. React Navigation's native stack uses Android's own screen transitions, so it feels like a normal app.

Each screen has a name and a set of **parameters** it needs. Details cannot work without knowing which title to show, so it gets `mediaType` and `tmdbId`. Home and Search need nothing. You write all of this down once in one type, the **param list**. Every screen's props are then typed from that list, so `navigation.navigate('Details', ...)` without the right parameters is a compile error.

Screens pass the TMDB id, not the whole title object. Details then loads what it needs itself. That keeps the parameters small and means a screen never shows stale data that was copied when you tapped.

React Navigation's dependencies include native code, and the docs ask for two small changes in the Android project. That makes this the last milestone where you touch `android/` and rebuild.

Where it fits

```
App.tsx
  └── SafeAreaProvider
        └── NavigationContainer
              └── RootNavigator (native stack, RootStackParamList)
                    ├── Home      params: none
                    ├── Search    params: none
                    └── Details   params: { mediaType, tmdbId }
```

New words

Term	Meaning
Component	A function that takes props and returns what to show.
Props	The inputs to a component, passed like attributes: `<PosterCard title={t} />`.
Navigator	A component that decides which screen is shown and handles moving between them.
Stack	Screens on top of each other. Back removes the top one.
Route	One screen entry in the navigator, with its name and parameters.
Param list	A type that maps each screen name to the parameters it takes (`undefined` for none).
Header	The bar at the top of a stack screen, with the title and the back arrow.
Safe area	The part of the screen not covered by the status bar, camera cut-out or navigation bar.

Steps

6.1 Install React Navigation and configure Android

Task. Install the React Navigation packages, make the two Android changes the Getting started page describes, and rebuild.

```
npm install @react-navigation/native @react-navigation/native-stack
npm install react-native-screens
npm run android
```

What this means. `@react-navigation/native` is the core and `native-stack` is the navigator you use. The navigator also needs two native libraries: `react-native-screens`, which you install now, and `react-native-safe-area-context`, which the template already installed. The docs ask you to set a fragment factory in `MainActivity.kt` (it prevents crashes when Android restores the app) and to turn off predictive back in `AndroidManifest.xml` (React Navigation does not support that gesture yet). Both are native changes, so you rebuild. When it works, the app builds and starts as before.

Shape.

- `android/app/src/main/java/com/mediatracker/MainActivity.kt`: add the `onCreate` override from the docs (Kotlin tab).
- `android/app/src/main/AndroidManifest.xml`: `android:enableOnBackInvokedCallback="false"` on the `<application>` tag.

Hint 1, direction. Follow the "Community CLI" and "Configuring react-native-screens on Android" parts of the Getting started page.

Hint 2, tools. Docs: https://reactnavigation.org/docs/getting-started. Search: "RNScreensFragmentFactory MainActivity", "enableOnBackInvokedCallback false".

Hint 3, plan. (1) Run the install command. (2) Open `MainActivity.kt` and add the override and its two imports as the docs describe. (3) Add the manifest attribute. (4) Rebuild.

Check. `npm run android` prints `BUILD SUCCESSFUL` and the app shows "Media Tracker". `git diff --stat` lists `MainActivity.kt`, `AndroidManifest.xml`, `package.json` and `package-lock.json`.

Common mistake. Editing the Java tab's code into a Kotlin file. The template's `MainActivity` is Kotlin (`.kt`).

6.2 The param list

Task. Write `RootStackParamList` in `src/navigation/types.ts`.

What this means. This type is the map of your app. Each key is a screen name, and its value is the parameter object that screen needs, or `undefined` if it needs none. The screens' props and every `navigate` call will be checked against it. When it works, the type check passes.

Shape.

- File: `src/navigation/types.ts`.
- `type RootStackParamList = { Home: undefined; Search: undefined; Details: { mediaType: MediaType; tmdbId: number } }` (exported).

Hint 1, direction. It is one object type. Import `MediaType` from the domain.

Hint 2, tools. Docs: https://reactnavigation.org/docs/typescript. Search: "react navigation param list typescript".

Hint 3, plan. (1) Import `MediaType`. (2) Write and export the type.

Check. `npm run typecheck` passes.

6.3 Three placeholder screens

Task. Create `HomeScreen`, `SearchScreen` and `DetailsScreen` in `src/screens/`. Each shows its own name, and Details also shows its parameters.

What this means. You start with placeholders so the navigation can be tested before any real screen exists. Each screen is a component that receives `navigation` and `route` props typed from your param list. Details reads `route.params` and shows, for example, `tv 1396`. TypeScript knows that `route.params` exists on Details and not on Home, because of the param list. When it works, the type check passes. You see the screens in the next step.

Shape.

- `src/screens/HomeScreen.tsx`: `HomeScreen(props: NativeStackScreenProps<RootStackParamList, 'Home'>): React.JSX.Element`
- `src/screens/SearchScreen.tsx`: same pattern with `'Search'`.
- `src/screens/DetailsScreen.tsx`: same pattern with `'Details'`. Shows `"<mediaType> <tmdbId>"`.
- Named exports (not default exports).

Hint 1, direction. Three small components, each typed with the generic props type and the screen's name.

Hint 2, tools. `NativeStackScreenProps` from `@react-navigation/native-stack`. Destructure props: `({ route })`. Docs: https://reactnavigation.org/docs/typescript#type-checking-screens. Search: "NativeStackScreenProps example".

Hint 3, plan. (1) Write `HomeScreen` with a centered `Text`. (2) Copy the structure for Search. (3) For Details, read `mediaType` and `tmdbId` from `route.params` and show them.

Check. `npm run typecheck` passes. Temporarily read `route.params` in `HomeScreen`: `tsc` reports an error, because Home has no parameters. Remove it.

6.4 The navigator and moving between screens

Task. Write `RootNavigator`, render it from `App.tsx` inside a safe area provider and navigation container, and add a search button at the top of Home plus a temporary button that opens Details for Breaking Bad.

What this means. `RootNavigator` creates the native stack from your param list and registers the three screens with header titles. `App.tsx` becomes only the wiring: providers around the navigator. On Home, a full-width button at the top (styled like a search bar, as in your design) navigates to Search. The temporary button proves that parameters arrive. When it works, you can move around and the phone's back button and back gesture take you back.

Shape.

- `src/navigation/RootNavigator.tsx`: `RootNavigator(): React.JSX.Element`. Header titles: Home "Media Tracker", Search "Search", Details "Details".
- `App.tsx`: the template's `SafeAreaProvider` (kept in step 2.6) → `NavigationContainer` → `RootNavigator`.
- Home: a `Pressable` at the top with the text "Search movies, series, anime" that opens Search.
- Temporary on Home: a button "Open Breaking Bad" → Details with `{ mediaType: 'tv', tmdbId: 1396 }`.

Hint 1, direction. Create the stack once at module level with your param list as the generic, then list the screens inside its navigator.

Hint 2, tools. `createNativeStackNavigator<RootStackParamList>()`, `Stack.Navigator`, `Stack.Screen` with `name`, `component`, `options={{ title }}`. `navigation.navigate(...)`. `Pressable`. Docs: https://reactnavigation.org/docs/hello-react-navigation (Dynamic), https://reactnavigation.org/docs/navigating, https://reactnative.dev/docs/pressable. Search: "react navigation native stack dynamic typescript".

Hint 3, plan. (1) In `RootNavigator.tsx`, create the stack and render three screens. (2) In `App.tsx`, replace the centered text with the providers and the navigator. (3) In Home, add the search `Pressable` and the temporary button, each calling `navigation.navigate`. (4) Style the search button so it looks like a search bar: full width, rounded, light background.

Check. On the phone: tapping the search bar opens a screen titled "Search" with a back arrow. The phone's back gesture returns to "Media Tracker". "Open Breaking Bad" opens Details showing `tv 1396`. `npm run check` passes.

Common mistake. Calling `createNativeStackNavigator` inside the component. It then makes a new navigator on every render.

Stuck?

If you see	Look at
App crashes right after start	Rebuild after installing (step 6.1), and the `MainActivity.kt` change.
`Couldn't find a navigation object`	`NavigationContainer` around the navigator in `App.tsx`.
`Argument of type ... is not assignable` on `navigate`	The parameters you pass against `RootStackParamList`.
Content under the status bar	`SafeAreaProvider` in `App.tsx`. Stack headers handle it, your own views may need `useSafeAreaInsets`.
Back gesture closes the app instead of going back	The manifest change in step 6.1.

Done when

- Three typed screens, a working stack and a working back button.
- Details shows the parameters it received.
- `npm run check` passes and the milestone is ticked in the README.
- Committed with a message that names the milestone (`Milestone 6: navigation`). The temporary "Open Breaking Bad" button stays until step 7.2 (it is the only way to reach Details until then).

Answer without looking

6.1. Why does Details receive `mediaType` and `tmdbId` instead of a whole `TrackedTitle`?

6.2. What does the param list let TypeScript check that it could not check before?

6.3. Why was a rebuild needed in step 6.1 but not in step 6.4?

6.4. In the stack Home → Search → Details, what does the back button show, and why?

MILESTONE 7

The main screen grid

Goal. Build the main screen: the Watching, Done and Watchlist filter and a three-column poster grid with progress badges, loaded from SQLite every time the screen is shown.

You will learn. useState · useCallback · useFocusEffect · FlatList with columns · Image from a URL · absolute positioning · loading and empty states · lifting state up

Before you start.

- React, State: a component's memory: https://react.dev/learn/state-a-components-memory
- React Native FlatList: https://reactnative.dev/docs/flatlist · Image: https://reactnative.dev/docs/image · Style: https://reactnative.dev/docs/style
- React Navigation, useFocusEffect: https://reactnavigation.org/docs/use-focus-effect
- TMDB attribution FAQ (logo and notice rules): https://developer.themoviedb.org/docs/faq
- Cheat sheet F (React Native components and hooks).

Briefing

The idea in plain words

A component re-runs (re-renders) whenever its **state** changes, and the screen updates to match. The Home screen keeps two pieces of state: which status is selected, and the list of titles for that status. Tapping a filter button changes the selected status. That triggers loading, loading sets the list, and the grid redraws. You never change the screen directly. You change state, and React draws the result.

The filter buttons do not keep their own state. Home owns the selected status and passes it down as a prop, together with a function to call when you tap. This is called **lifting state up**: the component that needs the value to load data is the one that owns it. The same `StatusFilter` component will be reused on the details page in Milestone 9.

When does Home load? Not only on start. When you add a title from Search and press back, Home must show it, but Home was never removed from the stack, so it does not start again. `useFocusEffect` runs your loading function every time the screen gains focus. It needs a stable function, which `useCallback` gives you. Without it, the effect would see a new function on every render and run in a loop.

The grid uses `FlatList`, which only draws the items that are visible, so it stays fast with hundreds of posters. Each poster is an `Image` with a remote URL from `posterUrl`. The badge sits on top of the poster in its corner with absolute positioning. Its content comes from `progressBadge`, which you already tested, so the component only draws what the function decided.

Where it fits

```
HomeScreen  (state: status, titles, loading)
  ├── search bar  ──> Search
  ├── StatusFilter  value=status  onChange=setStatus
  └── FlatList numColumns=3
        └── PosterCard  title  onPress ──> Details { mediaType, tmdbId }
              ├── Image  posterUrl(posterPath, 'w342')
              └── ProgressBadge  badge = progressBadge(title)
  on focus / status change:  getAppDb() → listTitlesByStatus(db, status)
```

New words

Term	Meaning
State	Values a component remembers between renders. Changing state re-renders the component.
Render	Running the component function to work out what the screen should show.
Hook	A function starting with `use` that gives a component a feature, like state or effects.

Term	Meaning
Effect	Code that runs after rendering to talk to the outside world, like loading from the database.
Focus	A screen has focus when it is the visible top of the stack.
Lifting state up	Keeping state in the parent and passing it to children as props.
Absolute positioning	Placing a view at fixed offsets inside its parent, on top of other content.
Empty state	What a list shows when it has no items.

Steps

7.1 The status filter

Task. Write `StatusFilter`, a row of three buttons (Watching, Done, Watchlist), and use it on Home with the selected status kept in Home's state.

What this means. The component draws three buttons and highlights the one equal to its `value` prop. Tapping a button calls `onChange` with that status. It does not remember anything itself. Home holds `status` in state, starting at `'watching'`, and passes it down. For now, Home also shows a temporary line "Showing: watching" so you can see the state change. When it works, tapping moves the highlight and changes that line.

Shape.

- File: `src/components/StatusFilter.tsx`.
- `StatusFilter(props: { value: Status; onChange: (status: Status) => void }): React.JSX.Element`
- Labels: Watching, Done, Watchlist, in that order. The selected button has a filled background, the others an outline.
- Home: `useState<Status>('watching')`, `StatusFilter` under the search bar. Temporary text "Showing: <status>" (remove at the end of step 7.2).

Hint 1, direction. A presentational component: props in, taps out. Build the three buttons from a list instead of writing them three times.

Hint 2, tools. `useState`, `Pressable`, `Array.prototype.map` in JSX with a `key`, conditional styles with an array: `style={[styles.button, selected && styles.selected]}`. Docs: https://react.dev/learn/state-a-components-memory, https://react.dev/learn/rendering-lists. Search: "react native segmented buttons pressable", "react native conditional style array".

Hint 3, plan. (1) Make a constant list of `{ status, label }` pairs. (2) Map it to `Pressable`s with `key` set to the status. (3) Pick the style by comparing with `value`. (4) Call `onChange(status)` on press. (5) In Home, add the state and render the filter and the temporary text.

Check. On the phone, Watching is highlighted at start. Tapping Done highlights Done and the text changes to "Showing: done". `npm run check` passes.

Common mistake. Forgetting the `key` prop. React warns "Each child in a list should have a unique key prop" in the Metro log.

7.2 Load the list on focus

Task. Load titles for the selected status with `listTitlesByStatus` whenever Home gains focus or the status changes. Show a loading text, an empty text, or the number of titles. Replace the temporary "Open Breaking Bad" button with a temporary "Add test titles" button.

What this means. This connects Milestone 5 to the screen. The loading function gets the database, queries one status and puts the result in state. It must re-run when the status changes, so `status` is in the dependency list of `useCallback`. `useFocusEffect` runs it when Home appears and every time you come back to it. Your database is empty, so the test button adds two titles directly: Breaking Bad as Watching and The Matrix as Done. Copy their `poster_path` values from your fixtures. For now, Home shows a count instead of a grid. When it works, the count follows the filter.

Shape.

- Home state: `titles: TrackedTitle[]` (start `[]`) and `loading: boolean` (start `true`).
- Loading function: `useCallback`, dependency list `[status]`, passed to `useFocusEffect`.
- Empty text: "Nothing here yet. Tap the search bar to add something."
- Temporary button "Add test titles": `addTitle` for Breaking Bad (1396, tv, totalEpisodes 62, watching) and The Matrix (603, movie, totalEpisodes 1, then `setStatus` to done), then reload. Remove it in step 8.6.
- Remove the "Showing: ..." text from step 7.1 and the "Open Breaking Bad" button from step 6.4.

Hint 1, direction. An async function inside a stable callback, run on focus. Set loading before, set the list after.

Hint 2, tools. `useFocusEffect` from `@react-navigation/native`, `useCallback` from `react`. Inside the callback, start an async function and call it (the callback itself must not be async). Docs: https://reactnavigation.org/docs/use-focus-effect, https://react.dev/reference/react/useCallback. Search: "useFocusEffect async function", "useCallback dependency array".

Hint 3, plan. (1) Add the two state values. (2) Write the callback: set loading, get the database, list by status, set titles, clear loading. (3) Pass it to `useFocusEffect`. (4) Render: loading text, or empty text, or "<n> titles". (5) Write the test button's press handler with the two adds. (6) Remove the old temporary pieces.

Check. Fresh install: all three filters show the empty text. After "Add test titles": Watching shows "1 titles", Done shows "1 titles", Watchlist shows the empty text. Pressing the button again keeps the counts at 1 (the UNIQUE rule). `npm run check` passes.

Common mistake. Making the `useFocusEffect` callback itself `async`. It must return nothing or a cleanup function, never a Promise.

7.3 The progress badge

Task. Write `ProgressBadge`, a small circle that shows a check mark, a percentage, or nothing.

What this means. This component draws exactly what `progressBadge` decided. It switches on `badge.kind`. For `'none'` it returns `null`, which in React means "draw nothing". For `'percent'` it shows the number with a `%` sign, and for `'check'` it shows a check mark. Because `Badge` is a discriminated union, TypeScript only lets you read `badge.value` inside the `'percent'` branch. When it works, you can see a badge on Home (temporarily) for the Breaking Bad title.

Shape.

- File: `src/components/ProgressBadge.tsx`.
- `ProgressBadge(props: { badge: Badge }): React.JSX.Element | null`
- A circle about 34 by 34, a solid accent color, white bold text. The check mark is the character ✓.

Hint 1, direction. A `switch` on the discriminant, one return per case.

Hint 2, tools. `switch (badge.kind)`, returning `null`, `borderRadius` equal to half the width for a circle. Docs: https://www.typescriptlang.org/docs/handbook/2/narrowing.html#discriminated-unions, https://reactnative.dev/docs/view-style-props. Search: "react native circle view", "react return null render nothing".

Hint 3, plan. (1) Switch on the kind. (2) None → return null. (3) Otherwise render a circular `View` with a `Text` inside: the value plus `%`, or the check mark. (4) Temporarily, on Home, render one `ProgressBadge` for the first title in the list using `progressBadge(...)`, and remove it after the check.

Check. With Watching selected, the temporary badge shows "0%". With Done selected, it shows ✓. Remove the temporary badge. `npm run check` passes.

Common mistake. Reading `badge.value` before checking `kind`. TypeScript reports that `value` does not exist on the other kinds.

7.4 The poster card

Task. Write `PosterCard`: the poster image with its badge in the bottom-right corner and the name under it, all pressable.

What this means. One card is one cell of the grid. The image comes from `posterUrl(title.posterPath, 'w342')`. A remote image has no size of its own in React Native, so you give it a width of 100% and a 2:3 aspect ratio, like a real poster. If there is no poster, you show a grey box of the same size with the name inside. The badge is absolutely positioned inside a wrapper around the image. The name is limited to two lines. When it works, you see the Breaking Bad poster with its badge.

Shape.

- File: `src/components/PosterCard.tsx`.
- `PosterCard(props: { title: TrackedTitle; onPress: () => void }): React.JSX.Element`
- Image: `posterUrl(title.posterPath, 'w342')`, `aspectRatio: 2 / 3`, rounded corners. `null` URL → grey placeholder `View` of the same shape.
- Badge: `ProgressBadge` with `progressBadge(title)`, absolute, 6 from the bottom and right edges.
- Name: `Text` with `numberOfLines={2}`.

Hint 1, direction. A pressable column: an image wrapper with something layered on top, then text.

Hint 2, tools. `Image` with `source={{ uri }}`, `aspectRatio`, `position: 'absolute'`, `bottom`, `right`, `numberOfLines`. Docs: https://reactnative.dev/docs/image, https://reactnative.dev/docs/layout-props#position. Search: "react native image aspectRatio width 100%", "react native overlay absolute bottom right".

Hint 3, plan. (1) Wrap everything in a `Pressable`. (2) Make an image wrapper `View` with `position: 'relative'`. (3) Inside it, the `Image` or the placeholder, then the badge with absolute position. (4) Under the wrapper, the name. (5) Temporarily render one card for the first title on Home, then remove it after the check.

Check. The temporary card shows Breaking Bad's poster at poster proportions with "0%" in the bottom-right corner and the name under it. Remove the temporary card. `npm run check` passes.

Common mistake. Giving the `Image` no size. A remote image without width and height is drawn 0 by 0, so you see nothing and get no error.

7.5 The grid

Task. Replace the count on Home with a three-column `FlatList` of `PosterCard`s. Tapping a card opens Details for that title.

What this means. `FlatList` takes your `titles` array as data and a function that draws one item. `numColumns={3}` puts three items per row. Each item needs a stable key, and the local database `id` is perfect. Three cards per row means each card is a third of the width minus some spacing. The empty text becomes the list's empty component. When you tap a card, Home navigates to Details with the title's `mediaType` and `tmdbId`. When it works, your two test titles appear as posters in their tabs, and tapping one opens Details with its ids.

Shape.

- Home: `FlatList` with `data={titles}`, `numColumns={3}`, key from `id`, `renderItem` drawing `PosterCard`.
- `ListEmptyComponent`: the empty text from step 7.2 (only when not loading).
- Card press → `navigation.navigate('Details', { mediaType: title.mediaType, tmdbId: title.tmdbId })`.
- Spacing: 12 around the grid, about 10 between cards.

Hint 1, direction. FlatList with columns: data, key, render function, and a style that gives each cell an equal share of the row.

Hint 2, tools. `FlatList` props `data`, `keyExtractor`, `renderItem`, `numColumns`, `columnWrapperStyle`, `contentContainerStyle`, `ListEmptyComponent`. Cell width: `flex: 1` plus `maxWidth: '33.33%'` so a last row with one item does not stretch. Docs: https://reactnative.dev/docs/flatlist. Search: "flatlist numColumns grid equal width", "flatlist last row stretched numColumns".

Hint 3, plan. (1) Replace the count with a `FlatList`. (2) Give each cell `flex: 1`, a maximum width of a third, and horizontal padding. (3) Render `PosterCard` with an `onPress` that navigates. (4) Move the empty text into `ListEmptyComponent`. (5) Add padding to the content container.

Check. Watching shows one Breaking Bad poster with "0%", left-aligned and one third wide. Done shows The Matrix with ✓. Tapping Breaking Bad opens Details showing `tv 1396`, and tapping The Matrix shows `movie 603`. `npm run check` passes.

Common mistake. Changing `numColumns` while the app runs. FlatList throws "Changing numColumns on the fly is not supported". Reload the app instead.

7.6 A Credits section for TMDB

Task. Add a small Credits section at the bottom of Home with an approved TMDB logo and TMDB's required notice.

What this means. TMDB's FAQ sets three rules for apps that use its API. You must show the notice sentence that says the app uses the TMDB API but is not endorsed or certified by TMDB. You must use one of TMDB's approved logos, and it must be less prominent than your own app's name. And the attribution must sit in an "About" or "Credits" type section. The grid's footer becomes that section: a heading "Credits", the logo and the sentence. A local image file is loaded with `require`, so it is bundled into the app and needs no internet. When it works, the Credits section is visible under the posters and under the empty text.

Shape.

- Logo file: an approved logo from https://www.themoviedb.org/about/logos-attribution, saved as `assets/tmdb-logo.png`. React Native's `Image` does not draw SVG files, so use a PNG (if only SVG is offered, convert it, for example with `rsvg-convert` from Arch's `librsvg` package).
- Home: `ListFooterComponent` with a small heading "Credits", the logo (about 20 high, smaller than the "Media Tracker" title) and the notice sentence from the FAQ in small grey text.

Hint 1, direction. Read the FAQ's attribution rules first, then build a small footer that meets each of them.

Hint 2, tools. `ListFooterComponent`, `Image` with `source={require('../../assets/tmdb-logo.png')}` (the path is relative to the file that uses it), `resizeMode="contain"`. Docs: https://developer.themoviedb.org/docs/faq, https://reactnative.dev/docs/images#static-image-resources. Search: "tmdb api attribution requirements", "react native require local image".

Hint 3, plan. (1) Read the attribution rules in the FAQ. (2) Download an approved logo and save it as a PNG in `assets/`. (3) Build the footer: heading, logo, notice. (4) Give the logo a fixed height and `resizeMode="contain"`. (5) Check it in all three tabs.

Check. Under the posters, and under the empty text in Watchlist, a Credits section shows the TMDB logo (smaller than the app title) and the notice sentence word for word as in the FAQ. `npm run check` passes.

Common mistake. Loading the logo from a web URL. It then disappears without internet, and a remote image needs an explicit size.

Stuck?

If you see	Look at
The list never updates after adding	`useFocusEffect` and the `[status]` dependency list.
The app freezes and the log repeats the same query	The callback is not wrapped in `useCallback`, so the effect runs on every render.
Posters are invisible but no error	Image size (step 7.4 common mistake), or `posterPath` is `null`.
One poster fills the whole row	`maxWidth` on the cell (step 7.5 Hint 2).
Badge appears under the image instead of on it	The wrapper must contain both, and the badge needs `position: 'absolute'`.
`Each child in a list should have a unique key prop`	`key` in `StatusFilter`, `keyExtractor` in `FlatList`.

Done when

- Home shows the filter and a three-column poster grid with correct badges.
- The grid reloads when you switch tabs and when you come back to Home.
- TMDB is credited in a Credits section with its logo and notice.
- `npm run check` passes and the milestone is ticked in the README.
- Committed with a message that names the milestone (`Milestone 7: main screen grid`). The "Add test titles" button stays until step 8.6.

Answer without looking

7.1. Why does Home own the selected status instead of `StatusFilter`?

7.2. Why is `useFocusEffect` needed, when `useEffect` would also run on start?

7.3. What would happen without `useCallback` around the loading function?

7.4. `ProgressBadge` returns `null` for some titles. Where was the decision made, and why there?

7.5. Why does `FlatList` draw faster than mapping 300 cards inside a `ScrollView`?

MILESTONE 8

Search and add

Goal. Search TMDB as you type, show the results, and add a title as Watching or Watchlist from its details page so it appears in the grid.

You will learn. controlled TextInput · debouncing · effect cleanup and stale responses · error messages from errors · discriminated unions for loaded data · custom hooks

Before you start.

- React Native TextInput: https://reactnative.dev/docs/textinput
- React, Synchronizing with Effects (cleanup and race conditions): https://react.dev/learn/synchronizing-with-effects
- React, Reusing Logic with Custom Hooks: https://react.dev/learn/reusing-logic-with-custom-hooks
- MDN setTimeout / clearTimeout: https://developer.mozilla.org/en-US/docs/Web/API/Window/setTimeout

Briefing

The idea in plain words

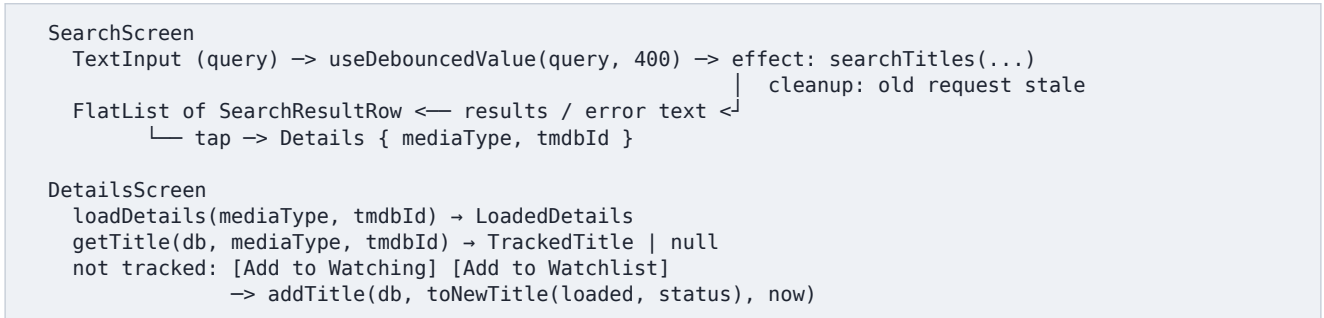
If the app searched on every key press, typing "breaking" would send eight requests, and TMDB has a rate limit. **Debouncing** means waiting until the user pauses: every key press restarts a short timer, and only when the timer runs out (here 400 ms) does the value change. You write this once as a **custom hook**, `useDebouncedValue`, a function that uses other hooks and can be reused in any component.

Requests finish in any order. If you type "mat", pause, then type "matrix", the "mat" answer might arrive after the "matrix" answer and overwrite it. An effect can return a **cleanup function** that React runs before the next run of the same effect. You use it to mark the old request as stale, and when a stale answer arrives you ignore it.

Errors need words a person understands. A `TmdbError` with status 401 means your token is wrong, other statuses mean TMDB has a problem, and anything else that `fetch` throws means no connection. A pure function turns any error into one sentence, so the screen does not need to know the details, and you can test it.

Adding happens on the details page, not in the search list. You open a result, see what it is, and then choose Watching or Watchlist. To save it, you need a `NewTitle`, and its `totalEpisodes` comes from the series details (a movie counts as 1). The details page holds either series details or movie details, so you describe "loaded details" as a discriminated union on `mediaType`. One pure function turns that into a `NewTitle`.

Where it fits



New words

Term	Meaning
Controlled input	A text field whose value lives in state and is updated on every change.
Debounce	Wait until changes stop for a moment, then act once.
Custom hook	Your own function starting with `use` that combines other hooks.
Cleanup function	A function returned from an effect. React runs it before the effect runs again or when the component goes away.
Stale response	An answer to a request that is no longer the latest one.
Race condition	A bug where the result depends on which of two operations finishes first.
Rate limit	A limit on how many requests you may send in a time window.

Steps

8.1 A debounced value hook

Task. Write `useDebouncedValue` and use it on the Search screen with a text input.

What this means. The hook takes a value and a delay. It keeps its own copy of the value in state. An effect starts a timer whenever the input value changes, and when the timer fires, it copies the value into its state. The cleanup function cancels the previous timer, so typing quickly keeps restarting it. The Search screen has a `TextInput` whose text lives in state (a controlled input). Temporarily, it shows the debounced value under the input. When it works, that line only changes after you stop typing.

Shape.

- File: `src/hooks/useDebouncedValue.ts`.
- `useDebouncedValue<T>(value: T, delayMs: number): T`
- Search screen: state `query: string`, `TextInput` with `autoFocus`, placeholder "Movie, series or anime", and `debounced = useDebouncedValue(query, 400)`.
- Temporary text under the input: "Searching for: <debounced>" (remove in step 8.3).

Hint 1, direction. State plus an effect with a timer, and cleanup that clears the timer.

Hint 2, tools. `useState`, `useEffect` with `[value, delayMs]`, `setTimeout`, `clearTimeout`. Generic functions with <T>. Docs: https://react.dev/learn/reusing-logic-with-custom-hooks, https://reactnative.dev/docs/textinput. Search: "react usedebounce hook settimeout cleanup", "typescript generic function".

Hint 3, plan. (1) Keep a state copy of the value. (2) In an effect, start a timer that sets the copy. (3) Return a cleanup that clears the timer. (4) Return the copy. (5) On Search, add the controlled input and the temporary text.

Check. Type "matrix" quickly: the temporary line stays empty while you type and shows "Searching for: matrix" about half a second after you stop. `npm run check` passes.

Common mistake. Leaving out the cleanup. Every key press then fires its own timer and the line flickers through "m", "ma", "mat"...

8.2 Errors in plain words

Task. Write `describeTmdbError` in `src/tmdb/client.ts`, with tests.

What this means. Search and Details both need to show a sentence when something goes wrong. The function gets whatever was thrown, which TypeScript types as `unknown`, and decides what kind of error it is. `instanceof TmdbError` checks whether it is your error class, and if so you look at the status. Anything else is treated as a connection problem. When it works, three tests pass.

Shape.

- `describeTmdbError(error: unknown): string` in `src/tmdb/client.ts`. Tests: `src/tmdb/client.test.ts`.
- Status 401 → `"TMDB rejected the token. Check src/config/secrets.ts."`
- Other `TmdbError` → `"TMDB had a problem (<status>). Try again later."`
- Anything else → `"Could not reach TMDB. Check your connection."`

Hint 1, direction. Narrow `unknown` with `instanceof`, then branch on the status.

Hint 2, tools. `instanceof`, template literal for the status. Docs: https://www.typescriptlang.org/docs/handbook/2/narrowing.html#instanceof-narrowing. Search: "typescript catch unknown instanceof".

Hint 3, plan. (1) If the error is a `TmdbError` with status 401, return the token sentence. (2) If it is any other `TmdbError`, return the status sentence. (3) Otherwise return the connection sentence. (4) Write one test per case.

Check. `npm test` passes: `new TmdbError(401)` → token sentence. `new TmdbError(500)` → `"TMDB had a problem (500). Try again later."`. `new TypeError('Network request failed')` → connection sentence.

Common mistake. Checking `error.status` without narrowing first. On `unknown`, TypeScript allows no property access.

8.3 Search as you type

Task. When the debounced query changes, call `searchTitles`, and keep the results, a loading flag and an error message in state. Ignore stale answers.

What this means. An effect depends on the debounced query. It sets loading, calls `searchTitles`, and stores either the results or the sentence from `describeTmdbError`. A local variable `cancelled` starts as `false`, and the cleanup function sets it to `true`. When an answer comes back after a newer query started, `cancelled` is true and the answer is thrown away. An empty query gives an empty list right away (your API function already does that). When it works, results appear after a pause and errors appear as sentences.

Shape.

- Search state: `results: SearchResult[]`, `loading: boolean`, `error: string | null`.
- Effect with dependency list `[debounced]`. Cleanup sets the request's `cancelled` flag.
- Under the input: "Searching…" while loading, the error in red, or (temporarily) the result names as plain `Text` lines.
- Remove the "Searching for:" text from step 8.1.

Hint 1, direction. An async function inside an effect, guarded by a flag that the cleanup flips.

Hint 2, tools. `useEffect` returning a cleanup, `try`/`catch`/`finally`. Docs: https://react.dev/learn/synchronizing-with-effects#fetching-data. Search: "useEffect fetch race condition ignore flag", "react effect cleanup cancelled".

Hint 3, plan. (1) In the effect, declare `cancelled = false`. (2) Define and call an async function: clear the error, set loading, await the search. (3) If not cancelled, store results. (4) In `catch`, if not cancelled, store the described error and clear results. (5) In `finally`, if not cancelled, clear loading. (6) Return a cleanup that sets `cancelled = true`.

Check. Type "breaking bad": after a pause, "Breaking Bad" is among the listed names. Turn on airplane mode, change the query: the red text says "Could not reach TMDB. Check your connection." Turn airplane mode off. `npm run check` passes.

Common mistake. Setting state after `await` without checking `cancelled`, so a slow old answer replaces the new one.

8.4 Search result rows

Task. Write `SearchResultRow` and show the results in a `FlatList`. Tapping a row opens Details.

What this means. Each row shows a small poster on the left (size `w185`), and on the right the name, the year and the type (Movie or Series). The list replaces the temporary plain names from step 8.3. Its key must combine type and id, because a movie and a series can have the same TMDB id. Tapping a row navigates to Details with the result's `mediaType` and `tmdbId`. When it works, search results look like a proper list and open their details page.

Shape.

- File: `src/components/SearchResultRow.tsx`.
- `SearchResultRow(props: { result: SearchResult; onPress: () => void }): React.JSX.Element`
- Poster: `posterUrl(result.posterPath, 'w185')`, width 56, `aspectRatio: 2 / 3`, grey placeholder if `null`.
- Second line: `"<year> · Movie"` or `"<year> · Series"`. Without a year: just the type.
- List key: `${mediaType}-${tmdbId}` . Hide the keyboard when the list is dragged.

Hint 1, direction. A horizontal row: image, then a column of two texts. The list is the same `FlatList` as before without columns.

Hint 2, tools. `flexDirection: 'row'`, `keyboardDismissMode="on-drag"` and `keyboardShouldPersistTaps="handled"` on the `FlatList`. Docs: https://reactnative.dev/docs/scrollview#keyboarddismissmode. Search: "react native list row image text", "keyboardShouldPersistTaps handled".

Hint 3, plan. (1) Write the row: `Pressable` with row layout, image or placeholder, then the texts. (2) Build the second line from year and type. (3) Replace the temporary names with a `FlatList` of rows. (4) Navigate on press.

Check. Searching "matrix" shows rows with posters, and The Matrix has "1999 · Movie". One tap on a row opens Details showing `movie 603`, even while the keyboard is open. `npm run check` passes.

Common mistake. Using only `tmdbId` as the key, which gives duplicate-key warnings for searches that return a movie and a series with the same id.

8.5 Loaded details and the new title

Task. Add `LoadedDetails` to the domain types, write `toNewTitle` in `src/domain/newTitle.ts` with tests, and write `loadDetails` in `src/tmdb/api.ts`.

What this means. The details page shows either a series or a movie. `LoadedDetails` says which, in one value: `{ mediaType: 'tv'; details: TvDetails }` or `{ mediaType: 'movie'; details: MovieDetails }`. `loadDetails` calls the right API function for a media type. `toNewTitle` builds what `addTitle` needs: a series uses its `totalEpisodes`, and a movie counts as 1. This is pure, so you test it with the mapped fixtures. When it works, two tests pass and the type check accepts all three pieces.

Shape.

- `src/domain/types.ts`: `LoadedDetails = { mediaType: 'tv'; details: TvDetails } | { mediaType: 'movie'; details: MovieDetails }`.
- `src/domain/newTitle.ts`: `toNewTitle(loaded: LoadedDetails, status: Status): NewTitle`. Tests: `src/domain/newTitle.test.ts`.
- `src/tmdb/api.ts`: `loadDetails(mediaType: MediaType, tmdbId: number): Promise<LoadedDetails>`.

Hint 1, direction. Another discriminated union. Switch on `mediaType` in both functions.

Hint 2, tools. Same tools as step 3.5. In the test, build `LoadedDetails` from `mapTvDetails(...)` and `mapMovieDetails(...)` on your fixtures. Search: "typescript switch exhaustive never".

Hint 3, plan. (1) Add the type. (2) In `toNewTitle`, pick `totalEpisodes` by media type and copy id, name and poster. (3) In `loadDetails`, call `getTvDetails` or `getMovieDetails` and wrap the result with its media type. (4) Write the tests.

Check. `npm test` passes: Breaking Bad's mapped fixture with status `'watching'` → `{ tmdbId: 1396, mediaType: 'tv', name: 'Breaking Bad', totalEpisodes: 62, status: 'watching', posterPath: <the fixture's poster_path> }` (use `toEqual`). The Matrix with `'watchlist'` → `totalEpisodes: 1`, `status: 'watchlist'`.

8.6 Add from the details page

Task. Make Details load the TMDB details and the local title. If the title is not tracked, show "Add to Watching" and "Add to Watchlist". Remove the "Add test titles" button from Home.

What this means. Details now does real work. On focus, it loads two things: the TMDB details (`loadDetails`) and your local row (`getTitle`). While loading it shows a spinner, and on error the sentence from `describeTmdbError`. It shows the poster and the name. If `getTitle` returned `null`, it shows the two add buttons. Pressing one calls `addTitle` with `toNewTitle(...)` and `Date.now()`, then reloads the local title so the buttons disappear. For now, a tracked title just shows its status as text. Milestone 9 builds the rest. When it works, you can add anything from search and find it in the grid.

Shape.

- Details state: `loaded: LoadedDetails | null`, `tracked: TrackedTitle | null`, `error: string | null`.
- Loading in `useFocusEffect`, both requests started together.
- Not tracked: two buttons. Tracked: text "In your list: <status>".
- Remove the "Add test titles" button and its handler from Home.

Hint 1, direction. Two independent loads that can run at the same time, then one render that depends on both.

Hint 2, tools. `Promise.all([...])`, `ActivityIndicator`, `useFocusEffect` + `useCallback` with `[route.params]` values as dependencies. Docs: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/all, https://reactnative.dev/docs/activityindicator. Search: "promise.all typescript tuple", "react native activityindicator center".

Hint 3, plan. (1) Write the load callback: get the database, then await both loads together. (2) Store both, or store the described error. (3) Render: spinner, error, or the content. (4) Content: poster (`w342`), name, and either the add buttons or the status text. (5) The add handler adds, then reloads. (6) Remove the test button from Home.

Check. Uninstall the app from the phone (to empty the database) and run `npm run android`. Search "breaking bad", open it, press "Add to Watching": the buttons change to "In your list: watching". Go back twice: Breaking Bad is in the Watching grid with "0%". Add The Matrix to Watchlist: it appears under Watchlist without a badge. `npm run check` passes.

Common mistake. Loading in a plain `useEffect`. After adding, then opening the same title again from the grid, it would still show old state until a full reload.

Stuck?

If you see	Look at
One request per key press in the log	The debounced value is not what the effect depends on, or the cleanup is missing.
Old results flash after new ones	The `cancelled` flag (step 8.3).
First tap on a result only closes the keyboard	`keyboardShouldPersistTaps="handled"` (step 8.4).
`Encountered two children with the same key`	The list key must combine media type and id.
Details stays on the spinner	An error thrown outside your `try`, or loading never cleared in `finally`.
Added title missing in the grid	Home's `useFocusEffect` (step 7.2), and the status you added with.

Done when

- Searching shows results after a short pause, with readable errors.
- Any title can be added as Watching or Watchlist and appears in the grid.
- No temporary buttons or texts remain.
- `npm run check` passes and the milestone is ticked in the README.
- Committed with a message that names the milestone (`Milestone 8: search and add`).

Answer without looking

8.1. What problem does debouncing solve, and why 400 ms instead of 0 or 3000?

8.2. Describe the race condition that the `cancelled` flag prevents.

8.3. Why does `describeTmdbError` take `unknown` instead of `TmdbError`?

8.4. Why is adding done on Details and not directly from the search list?

8.5. Why does a movie count as 1 episode in `toNewTitle`?

MILESTONE 9

Details and tracking

Goal. Finish the details page: change status, browse seasons, mark episodes, see your progress and the next episode date, and move finished titles to Done.

You will learn. FlatList with a header · Set for fast lookups · derived values · reloading after writes · pure rules for status changes · date formatting

Before you start.

- FlatList ListHeaderComponent: https://reactnative.dev/docs/flatlist#listheadercomponent
- MDN Set: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set
- React, Choosing the State Structure (avoid redundant state): https://react.dev/learn/choosing-the-state-structure

Briefing

The idea in plain words

The details page is the heart of the app, and it is long: header, status buttons, season picker, and up to dozens of episodes. A `ScrollView` with a `FlatList` inside triggers a warning and breaks scrolling, so the whole page becomes one `FlatList` of episodes, and everything above the episodes goes into its `ListHeaderComponent`.

For every episode row, the page must know "is this one watched?". Searching an array for every row is slow and clumsy. Instead you turn the watched list into a **Set** of keys like `"2:5"`. A set answers "do you contain this?" instantly. The key function is pure and tested, so the same episode always gets the same key.

Keep state small. The label "S2 E5" and the percentage are not stored in state: they are **derived** from the watched list and the totals every render, with the functions you wrote in Milestone 3. After every write (mark, unmark, status change) the page reloads the local title and the watched list from the database. That is simpler than updating several pieces of state by hand, and it can never disagree with what is stored.

Two small rules also become pure functions. Marking an episode on a Watchlist title means you started watching it, so it moves to Watching. And the next episode line only makes sense for a date that has not passed yet. Both get tests. Finally, totals change: when a new season of a show you watch is announced, TMDB's episode count grows. Every time Details opens a tracked series, it compares TMDB's total with the stored one and updates it if they differ, so the percentage in the grid stays honest.

Where it fits

```
DetailsScreen = FlatList (data: episodes of the selected season)
  ListHeaderComponent
    ├── poster, name, overview
    ├── StatusFilter  value=tracked.status  onChange → setStatus
    ├── progress line   progressLabel(latestEpisode(watched)) · progressPercent(...)
    ├── next line       formatNextEpisode(details.nextEpisode, today)
    ├── [Mark as Done] / movie: [Mark as watched]
    └── season picker   S1 S2 S3 ...  → getSeason(tmdbId, n)
  renderItem: EpisodeRow  watched = watchedKeys.has(episodeKey(ep))
                onToggle → markEpisode / unmarkEpisode → reload
```

New words

Term	Meaning
List header	Content drawn above the first item of a `FlatList`, scrolling with it.
Set	A collection of unique values with a fast `has` check.
Derived value	A value calculated from state during render instead of stored in state.
Redundant state	State that could be calculated from other state. It tends to fall out of sync.
ISO date	A date written as `YYYY-MM-DD`, the format TMDB uses.

Steps

9.1 Header, status and fresh totals

Task. Turn Details into a `FlatList` with a header, show the overview and a `StatusFilter` for tracked titles, and keep `totalEpisodes` up to date with a new `updateTotalEpisodes` query.

What this means. The page's structure changes now so the episodes can be added below. The header shows the poster, name and overview (four lines). For a tracked title it shows the same `StatusFilter` as Home, set to the title's status. Changing it calls `setStatus` and reloads. When a tracked series loads, compare its stored `totalEpisodes` with TMDB's and update the stored number if they differ. That query is new and tested. When it works, changing the status here moves the title between tabs on Home.

Shape.

- `src/db/titles.ts`: `updateTotalEpisodes(db: Db, id: number, totalEpisodes: number): Promise<void>`. Test in `titles.test.ts`.
- Details: `FlatList` with `data={[]}` for now and everything else in `ListHeaderComponent`.
- Header for a tracked title: `StatusFilter` with `value={tracked.status}`. Untracked titles keep the add buttons from step 8.6.
- The "In your list: <status>" text from step 8.6 is removed.

Hint 1, direction. Reuse: the same filter component, the same reload function, one new UPDATE.

Hint 2, tools. `ListHeaderComponent`, `numberOfLines`. SQL `UPDATE titles SET total_episodes = ? WHERE id = ?`. Search: "flatlist listheadercomponent scroll whole page".

Hint 3, plan. (1) Write and test `updateTotalEpisodes`. (2) Move the details content into a header component for a `FlatList`. (3) Add the overview. (4) Render the filter for tracked titles, with `onChange` doing `setStatus` and reload. (5) In the load function, after both loads, if it is a tracked series and the totals differ, update and reload.

Check. `npm test` passes: after `updateTotalEpisodes(db, id, 70)`, `getTitle` gives `totalEpisodes: 70`. On the phone, set Breaking Bad to Watchlist on its details page, go back: it is in the Watchlist tab, not Watching. Set it back to Watching. `npm run check` passes.

9.2 Season picker and episode list

Task. Show a row of season buttons (S1, S2, ...) for series. The selected season's episodes are loaded with `getSeason` and listed under the header.

What this means. TMDB does not send all episodes with the series details, so you load one season at a time. The selected season starts at the season of your latest watched episode, or 1 if you have not started. Tapping a season button loads that season into state, and the `FlatList`'s data becomes those episodes. For now, each episode is a plain `Text` line "E<n> · <name>". Movies have no seasons, so they show no picker and an empty list. When it works, switching seasons swaps the episode list.

Shape.

- Details state: `selectedSeason: number | null`, `episodes: Episode[]`, `seasonLoading: boolean`.

- Season buttons from `details.seasons` (already without specials), in a horizontal `ScrollView`, selected one filled.
- Starting season: `latestEpisode(watched)?.seasonNumber ?? 1`. The watched list comes from `listWatched` (load it with the local title).
- `FlatList` key: `${seasonNumber}-${episodeNumber}` .

Hint 1, direction. A second, smaller load that depends on the selected season, with its own loading flag.

Hint 2, tools. Horizontal `ScrollView`, `useEffect` with `[selectedSeason]`, the `cancelled` pattern from step 8.3, `??` (nullish coalescing). Docs: https://reactnative.dev/docs/scrollview#horizontal. Search: "react native horizontal scroll buttons", "javascript nullish coalescing".

Hint 3, plan. (1) Load `listWatched` together with the local title and keep it in state. (2) When details arrive for a series, set the starting season. (3) An effect loads the selected season's episodes, guarded by `cancelled`. (4) Render the season buttons in the header. (5) Feed the episodes to the `FlatList` with simple text rows.

Check. Breaking Bad opens on S1 with 7 episodes, the first "E1 · Pilot". Tapping S5 shows 16 episodes. The Matrix shows no season buttons. `npm run check` passes.

Common mistake. Loading the season inside the main focus loader, so every status change reloads the whole season too.

9.3 Mark episodes as watched

Task. Write `episodeKey` with a test, write `EpisodeRow` with a watched toggle, and connect it to `markEpisode` and `unmarkEpisode`.

What this means. `episodeKey` turns an episode reference into a string like `"2:5"`. The page builds a `Set` of keys from the watched list on each render, so each row can ask `has` instantly. `EpisodeRow` shows the episode number, name and air date, and a round toggle on the right that is filled when watched. Tapping the toggle marks or unmarks the episode with `Date.now()`, then reloads. Episodes can only be marked on tracked titles. Untracked ones show the toggle disabled. When it works, marks survive leaving the page and restarting the app.

Shape.

- `src/domain/progress.ts`: `episodeKey(ref: EpisodeRef): string` → `"<season>:<episode>"`. Test in `progress.test.ts`.
- File: `src/components/EpisodeRow.tsx`. `EpisodeRow(props: { episode: Episode; watched: boolean; disabled: boolean; onToggle: () => void }): React.JSX.Element`
- Row: "E<n>" in bold, the name (one line), the air date in grey if not `null`, and the toggle (✓ in a filled circle when watched, an empty circle when not).

Hint 1, direction. A set built from keys, a row that only reports taps, and a handler that writes and reloads.

Hint 2, tools. `new Set(array.map(...))`, `set.has(key)`, `Pressable` with `disabled`, `hitSlop` to make a small toggle easy to tap. Docs: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set, https://reactnative.dev/docs/pressable#hitslop. Search: "javascript set has", "react native checkbox circle pressable".

Hint 3, plan. (1) Write and test `episodeKey`. (2) Write `EpisodeRow`. (3) In Details, build the set from the watched list during render. (4) `renderItem` passes `watched` from the set. (5) The toggle handler marks or unmarks based on the current `watched`, then reloads the local title and watched list.

Check. `npm test` passes: `episodeKey({ seasonNumber: 2, episodeNumber: 5 })` → `"2:5"`. On the phone: mark S1 E1 and E2 of Breaking Bad, go back: the grid badge shows "3%" (2 of 62). Close the app fully and reopen: both are still marked. Unmark E2: the badge shows "2%". `npm run check` passes.

Common mistake. Keeping a separate `watched: boolean` state inside `EpisodeRow`. It then disagrees with the database after a reload.

9.4 Progress line and the Watchlist rule

Task. Write `statusAfterMark` with tests, apply it when marking, and show the progress line on Details.

What this means. If you mark an episode of something on your Watchlist, you have started it, so it should move to Watching. `statusAfterMark` holds that rule: Watchlist becomes Watching, everything else stays as it is (marking an extra episode of a Done show keeps it Done). The mark handler calls `setStatus` when the rule changes the status. The progress line shows where you are and how much is done, for example "S1 E2 · 3%", using functions from Milestone 3. It is derived, not stored. When it works, marking a Watchlist series moves it to Watching and the line updates on each tap.

Shape.

- `src/domain/progress.ts`: `statusAfterMark(current: Status): Status`. Tests in `progress.test.ts`.
- Progress line (tracked series only): `"<progressLabel(...)> · <progressPercent(...)>%"`. Not started: `"Not started · 0%"`.

Hint 1, direction. A three-case rule as a pure function, and a line computed during render.

Hint 2, tools. Reuse `latestEpisode`, `progressLabel`, `progressPercent`. Use `tracked.watchedCount` and `tracked.totalEpisodes`. Search: "react derived state avoid useState".

Hint 3, plan. (1) Write and test `statusAfterMark`. (2) In the mark handler (not unmark), after marking, compute the new status and call `setStatus` only if it changed. (3) Compute the progress text during render and show it under the status filter.

Check. `npm test` passes: `'watchlist'` → `'watching'`, `'watching'` → `'watching'`, `'done'` → `'done'`. On the phone: add a new series to Watchlist, mark S1 E1: the filter on Details switches to Watching, and the line shows "S1 E1 · <n>%". `npm run check` passes.

9.5 Next episode date

Task. Write `formatNextEpisode` with tests and show its result on Details for tracked series.

What this means. TMDB tells you the next episode that will air for running shows. The function turns it into a short line such as "Next: S3 E4 · 18 Oct". It returns `null` when there is no next episode, or when the air date is before today (TMDB sometimes lags a day or two). Today's date is a parameter, as a `YYYY-MM-DD` string, so the tests do not depend on the real date. Strings in that format compare correctly as text. The month names are short English names. When it works, a running show shows its next date and Breaking Bad shows nothing.

Shape.

- `src/domain/progress.ts`: `formatNextEpisode(next: NextEpisode | null, today: string): string | null`. Tests in `progress.test.ts`.
- Output: `"Next: S<season> E<episode> · <day> <Mon>"`, day without a leading zero. Months: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec.
- Air date before `today` → `null`. Air date equal to `today` → a line (it airs today).
- On Details, compute today from `new Date()` in local time as `YYYY-MM-DD`.

Hint 1, direction. Compare ISO date strings as text, then split the date and look up the month name.

Hint 2, tools. `String.prototype.split('-')`, `Number(...)`, an array of month names indexed by month − 1 (remember `noUncheckedIndexedAccess`). For today: `getFullYear`, `getMonth`, `getDate`, `String(n).padStart(2, '0')`. Search: "compare iso date strings javascript", "javascript padStart".

Hint 3, plan. (1) If `next` is null, return null. (2) If its air date is less than today, return null. (3) Split the air date into year, month, day. (4) Look up the month name, and return null if the lookup fails. (5) Build the text. (6) Write a small helper in Details for today's string.

Check. `npm test` passes: `({ seasonNumber: 3, episodeNumber: 4, airDate: '2026-10-18' }, '2026-10-12')` → `"Next: S3 E4 · 18 Oct"`. Same with air date `'2026-10-01'` → `null`. Air date `'2026-10-12'` with today `'2026-10-12'` → `"Next: S3 E4 · 12 Oct"`. `(null, '2026-10-12')` → `null`. On the phone, a running series you add shows a "Next:" line, and Breaking Bad does not.

Common mistake. Using `new Date('2026-10-18')` to read the day. That string is read as midnight UTC, which can be the previous day in some time zones.

9.6 Done, and movies

Task. Add a "Mark as Done" button for tracked series that are not done, write `markMovieWatched` with a test, and give movies a "Mark as watched" button.

What this means. When you finish a series you move it to Done yourself. The app does not do it automatically when you reach 100%, because a new season may come. The button calls `setStatus` with `'done'`. A movie has no episodes, so watching it is one action: it becomes Done and its `last_watched_at` is set, so it appears first in the Done grid. That needs one new query. After this step, every part of the summary you agreed on at the start works. When it works, finished titles appear under Done with a check mark.

Shape.

- `src/db/titles.ts`: `markMovieWatched(db: Db, id: number, now: number): Promise<void>` sets `status = 'done'` and `last_watched_at = now`. Test in `titles.test.ts`.
- Tracked series with status other than done: button "Mark as Done".
- Tracked movie with status other than done: button "Mark as watched". Done movies show "Watched" as text instead.

Hint 1, direction. One UPDATE with two columns, and two buttons that only appear in the right situation.

Hint 2, tools. Conditional rendering with `&&` or a ternary. Search: "react native conditional render button".

Hint 3, plan. (1) Write and test `markMovieWatched`. (2) In the header, decide which button to show from `loaded.mediaType` and `tracked.status`. (3) Each button writes, then reloads. (4) Go through the full flow on the phone once.

Check. `npm test` passes: add a movie with status watchlist at now 100, call `markMovieWatched(db, id, 500)`: `getTitle` gives `status: 'done'` and `lastWatchedAt: 500`. On the phone: "Mark as watched" on The Matrix moves it to the Done tab with ✓. "Mark as Done" on Breaking Bad moves it to Done with ✓. `npm run check` passes.

Stuck?

If you see	Look at
`VirtualizedLists should never be nested inside plain ScrollViews`	Details must be one `FlatList` with a header (step 9.1).
Toggling one episode re-renders slowly	Building the `Set` once per render is fine. Building it once per row is not.
Badge in the grid does not change after marking	Home's focus reload (step 7.2) and whether marking reached the database (step 5.6 tests).
Next date one day off	`new Date(string)` and time zones (step 9.5 common mistake).

If you see	Look at
Percentage over 100 after a series got shorter on TMDB	`progressPercent` clamps at 100. Check `updateTotalEpisodes` ran (step 9.1).
Season list keeps the old season after switching titles	Reset `selectedSeason` and `episodes` when the route's `tmdbId` changes.

Done when

- Status can be changed from Details, and the grid follows.
- Episodes can be marked and unmarked per season, and survive restarts.
- Progress, the Watchlist rule and the next episode line work and are tested.
- Movies and series can be finished and appear under Done.
- `npm run check` passes and all nine milestones are ticked in the README.
- Committed with a message that names the milestone (`Milestone 9: details and tracking`).

Answer without looking

9.1. Why is the whole details page one `FlatList` instead of a `ScrollView` with a list inside?

9.2. Why turn the watched list into a `Set` of string keys?

9.3. Why is the progress text not kept in state?

9.4. Why does the app not move a series to Done automatically at 100%?

9.5. Why does `formatNextEpisode` take today's date as a parameter?

Cheat sheet A: The Android toolchain

Which piece does what, and where it lives on your computer.

Piece	Lives in	Used by	You notice it when
Node.js + npm	pacman (`/usr/bin`)	CLI, Metro, Jest, ESLint, Prettier	`npm` commands fail or warn about the version
JDK 17	`/usr/lib/jvm/java-17-openjdk`	Gradle	build errors that mention Java versions
Gradle	downloaded by `android/gradlew`	`npm run android`	the first build takes long
cmdline-tools (Android CLI `android`, deprecated `sdkmanager`)	`~/Android/Sdk/cmdline-tools/latest`	you (`android sdk ...`)	`android: command not found`, or `android info` shows another SDK
`platforms/android-<compileSdkVersion>`	`~/Android/Sdk/platforms`	Gradle compile	Gradle names a missing platform
`build-tools/<buildToolsVersion>`	`~/Android/Sdk/build-tools`	Gradle packaging	Gradle names missing build tools
`ndk/<ndkVersion>` + CMake	`~/Android/Sdk/ndk`, `.../cmake`	C++ parts of React Native and op-sqlite	NDK or CMake errors during the build
platform-tools (adb)	`~/Android/Sdk/platform-tools`	install, logs, port forwarding	`adb devices` shows nothing
Metro	`node_modules`	serving your code to the app	red screen "Unable to load script"

When you change...	...do this
A `.ts` or `.tsx` file	Save. Fast Refresh updates the phone.
Anything in `android/`, or you installed a package with native code	`npm run android` again.
`package.json` dependencies (JavaScript only)	Restart Metro (`npm start -- --reset-cache` if it acts strange).
`tsconfig.json`	Restart the TS server in Neovim (`:LspRestart`).

Cheat sheet B: TypeScript

Small syntax examples. They use made-up names on purpose, so they do not solve any step.

Concept	Example	Notes
Type alias	`type Color = 'red' | 'blue';`	A name for a type.
Object type	`type Point = { x: number; y: number };`	Every field is required unless marked.
Nullable field	`label: string | null`	The value is always present, but may be `null`.
Optional field	`label?: string`	The field may be missing (`undefined`). Use for data you do not control.
Intersection	`type Point3 = Point & { z: number };`	Combines two object types.
Discriminated union	`type Shape = { kind: 'circle'; r: number } | { kind: 'square'; side: number };`	Check `shape.kind` and TypeScript knows which fields exist.

Concept	Example	Notes
Readonly array	`function f(items: readonly number[])`	`items.push` and `items.sort` become errors.
Generic function	`function first<T>(list: readonly T[]): T | undefined`	`T` is filled in from the argument.
`unknown`	`const data: unknown = JSON.parse(text);`	Safe: you must narrow before use. `any` turns checking off.
Type assertion	`const p = data as Point;`	You promise the type. Use at a trust boundary only.
Type guard	`function isColor(v: unknown): v is Color`	Returning true narrows `v` to `Color` in the caller.
`instanceof`	`if (err instanceof RangeError) { ... }`	Narrows to a class.
`Pick` / `Omit`	`type XY = Pick<Point3, 'x' | 'y'>;`	Build a type from some fields of another.
`Partial`	`Partial<Point>`	Every field optional. Handy for test helpers.
`Record`	`Record<string, number>`	An object with string keys and number values.
`noUncheckedIndexedAccess`	`const a = list[0]; // number | undefined`	Forces a check before using an indexed value.
Nullish coalescing	`value ?? fallback`	Uses the fallback only for `null` or `undefined`, not for `0` or `''`.
Optional chaining	`maybe?.name`	`undefined` if `maybe` is `null` or `undefined`.

Cheat sheet C: Jest

All test files end in `.test.ts` and sit next to the file they test.

Piece	Example	Meaning
Group	`describe('area', () => { ... })`	Groups related tests in the output.
Test	`it('adds two numbers', () => { ... })`	One case. `test(...)` is the same thing.
Equal value	`expect(sum(1, 2)).toBe(3)`	Strict equality. Use for numbers, strings, booleans, null.
Equal content	`expect(point).toEqual({ x: 1, y: 2 })`	Compares objects and arrays by content.
Null	`expect(found).toBeNull()`	Exactly `null`.
Length	`expect(list).toHaveLength(3)`	For arrays and strings.
Contains	`expect(list).toContainEqual({ x: 1, y: 2 })`	One element equal to this.
Async	`it('loads', async () => { expect(await load()).toBe(1); })`	Mark the test `async` and `await` inside.
Rejects	`await expect(promise).rejects.toThrow()`	The Promise fails with an error.
Before each	`beforeEach(async () => { ... })`	Runs before every test in the file or `describe`. Use it for a fresh database.
Run one file	`npx jest src/domain/progress.test.ts`	Faster while working on one area.
Watch mode	`npm test -- --watch`	Reruns tests for changed files on save.

Cheat sheet D: HTTP and TMDB

Base URL `https://api.themoviedb.org/3`. Every request sends `Authorization: Bearer <API Read Access Token>`.

Endpoint	Used for	Fields this app uses
`GET /search/multi?query=...`	Search	`results[].media_type` ('movie' | 'tv' | 'person'), `id`, `title` (movie) / `name` (tv), `release_date` / `first_air_date`, `poster_path`
`GET /tv/{series_id}`	Series details	`id`, `name`, `overview`, `poster_path`, `seasons[]` (`season_number`, `name`, `episode_count`), `next_episode_to_air` (`season_number`, `episode_number`, `air_date`) or null
`GET /tv/{series_id}/season/{season_number}`	Episodes of one season	`episodes[]` (`season_number`, `episode_number`, `name`, `air_date`)
`GET /movie/{movie_id}`	Movie details	`id`, `title`, `overview`, `poster_path`, `runtime`, `release_date`

Status	Meaning	This app says
200	OK, body is the data	(shows the data)
401	Token missing or wrong	TMDB rejected the token...
404	No such id	TMDB had a problem (404)...
429	Too many requests	TMDB had a problem (429)...
(fetch throws)	No network, DNS, TLS	Could not reach TMDB...

Image URLs: `https://image.tmdb.org/t/p/` + size + `poster_path`. Sizes used here: `w185` (search rows), `w342` (grid and details), `w500` (spare). Season 0 is "Specials" and is ignored everywhere in this app. Anime are ordinary TV series.

Cheat sheet E: SQL and SQLite

Examples use a made-up `books` and `notes` schema, not the app's tables.

Concept	Example	Notes
Create table	`CREATE TABLE books (id INTEGER PRIMARY KEY, title TEXT NOT NULL)`	`INTEGER PRIMARY KEY` gets a new number automatically.
CHECK	`kind TEXT NOT NULL CHECK (kind IN ('paper', 'ebook'))`	The database refuses other values.
UNIQUE pair	`UNIQUE (isbn, edition)`	No two rows with the same pair.
Foreign key	`book_id INTEGER NOT NULL REFERENCES books(id) ON DELETE CASCADE`	Needs `PRAGMA foreign_keys = ON` per connection.
Composite PK	`PRIMARY KEY (book_id, page)`	The pair identifies a row.
Placeholders	`SELECT * FROM books WHERE id = ?` with params `[7]`	Always. Never build SQL with values in the string.
Ignore duplicates	`INSERT OR IGNORE INTO notes (...) VALUES (?, ?)`	A conflicting row is skipped silently.
Upsert, do nothing	`INSERT INTO books (...) VALUES (...) ON CONFLICT (isbn, edition) DO NOTHING`	Same idea, naming the conflict.
Update	`UPDATE books SET title = ? WHERE id = ?`	Without `WHERE`, every row changes.
Delete	`DELETE FROM notes WHERE book_id = ? AND page = ?`	

Concept	Example	Notes
Left join + count	`SELECT b.id, COUNT(n.book_id) AS note_count FROM books b LEFT JOIN notes n ON n.book_id = b.id GROUP BY b.id`	Books with no notes get 0.
Fallback value	`COALESCE(read_at, bought_at)`	First non-null argument.
Sort	`ORDER BY a DESC, b ASC`	Several keys in order.
Schema version	`PRAGMA user_version` / `PRAGMA user_version = 2`	No ? allowed in PRAGMA.
List tables	`SELECT name FROM sqlite_master WHERE type = 'table'`	

Cheat sheet F: React Native components and hooks

Imports: components from `react-native`, hooks from `react`.

Piece	What it is for	Key props or rules
`View`	A box for layout	`style`. Layout is flexbox, column by default.
`Text`	All text must be inside one	`numberOfLines`, `style`
`Image`	Pictures	`source={{ uri }}` needs a size (`width` + `aspectRatio`)
`Pressable`	Anything tappable	`onPress`, `disabled`, `hitSlop`, `style`
`TextInput`	Text entry	`value`, `onChangeText`, `placeholder`, `autoFocus`
`FlatList`	Long lists, drawn lazily	`data`, `renderItem`, `keyExtractor`, `numColumns`, `ListHeaderComponent`, `ListEmptyComponent`, `ListFooterComponent`
`ScrollView`	Short scrolling content	`horizontal`. Never put a `FlatList` inside one.
`ActivityIndicator`	Spinner	`size`
`StyleSheet.create`	Define styles once	`const styles = StyleSheet.create({ box: { padding: 8 } })`
`useState`	Remember a value	`const [count, setCount] = useState(0)`. Setting it re-renders.
`useEffect`	Talk to the outside after render	Runs when the values in its dependency list change. May return a cleanup function.
`useCallback`	Keep the same function between renders	Changes only when its dependency list changes.
`useFocusEffect`	Run when a screen gains focus	From `@react-navigation/native`. Pass a `useCallback` function, not an async one.
Rules of hooks	Call hooks at the top of a component	Never inside `if`, loops or nested functions.

Layout keys you will use: `flex: 1` (take the free space), `flexDirection: 'row'`, `justifyContent` (along the main axis), `alignItems` (across it), `gap`, `padding`, `margin`, `position: 'absolute'` with `top/right/bottom/left`, `borderRadius`, `aspectRatio`.

Cheat sheet G: React Navigation

Version 7, native stack, dynamic configuration.

Piece	Example shape	Meaning
Param list	`type ParamList = { Inbox: undefined; Message: { id: number } }`	Screen names and their parameters.
Create the stack	`const Stack = createNativeStackNavigator<ParamList>();`	Once, at module level.
Register a screen	`<Stack.Screen name="Inbox" component={InboxScreen} options={{ title: 'Inbox' }} />`	Inside `<Stack.Navigator>`.
Screen props type	`NativeStackScreenProps<ParamList, 'Message'>`	Gives typed `navigation` and `route`.
Open a screen	`navigation.navigate('Message', { id: 3 })`	Pushes it on the stack (or goes back to it if it is already there).
Read parameters	`route.params.id`	Only on screens that have parameters.
Go back	`navigation.goBack()`	Same as the back button.
Run on focus	`useFocusEffect(useCallback(() => { ... }, [deps]))`	Runs each time the screen becomes visible.

Debugging table

Symptom	Likely cause	Fix or what to inspect
`SDK location not found`	`ANDROID_HOME` not set in this terminal	`echo $ANDROID_HOME`, open a new terminal (step 1.3).
Build errors that mention Java or class file versions	Gradle runs on another JDK than 17	`java -version`, `archlinux-java set`, `JAVA_HOME`.
Gradle names a missing SDK platform, build-tools or NDK	Installed versions differ from `android/build.gradle`	`android sdk list` against `compileSdkVersion`, `buildToolsVersion`, `ndkVersion` (step 2.3).
`No connected devices!`	Phone not visible to adb	`adb devices`. Cable, USB debugging, `adbusers` group, prompt on the phone.
Red screen "Unable to load script"	App cannot reach Metro	Is `npm start` running? `adb reverse tcp:8081 tcp:8081`, then reload (press R twice in Metro).
App closes immediately after start	Native library added without rebuild, or `MainActivity` change wrong	`npm run android`. `adb logcat *:E` for the native error.
Changes do not show up	Metro cache or a stale bundle	Reload. If still old: stop Metro, `npm start -- --reset-cache`.
Build fails after a new native package	Old native build output	`(cd android && ./gradlew clean)`, then `npm run android`.
`tsc` passes but Neovim shows errors (or the reverse)	Neovim started outside the repo, or old server state	Start Neovim in `MediaTracker`, `:LspRestart`.
Test fails: "Cannot find module"	Wrong relative path or file name case	Paths in imports are case-sensitive on Linux.
TMDB returns 401	Wrong token type or header	Long API Read Access Token, header `Authorization: Bearer <token>`.
`Network request failed`	No internet on the phone, or URL typo	Open the URL in the phone's browser. Check `https://`.
SQL constraint error in a test	Test data breaks a rule in the schema	Read which constraint. The database is doing its job.
List shows old data after going back	Loading in `useEffect` instead of `useFocusEffect`	Step 7.2 pattern.
App freezes and logs the same query forever	Effect callback recreated every render	`useCallback` and its dependency list.
Warning about nested VirtualizedLists	`FlatList` inside `ScrollView`	One `FlatList` with `ListHeaderComponent` (step 9.1).
Poster space is empty	Image without size, or `null` path	`aspectRatio` and width on the image. Log the URL from `posterUrl`.
Percent badge wrong	Stale `total_episodes` or wrong count	`updateTotalEpisodes` ran? Compare with `SELECT * FROM watched_episodes WHERE title_id = ?` in a test.

Answer key

Milestone 1: A clean Android toolchain

1.1 Why must the contents of the command-line tools zip end up in `cmdline-tools/latest` and not directly in `cmdline-tools`?

Google's setup instructions require that layout: the tools expect to live at `<sdk root>/cmdline-tools/latest/`, and the old `sdkmanager` works out the SDK root from that position. Unpacked one level too high, the tools are not where the SDK expects them and the SDK root comes out wrong.

1.2 Why does this milestone install only `platform-tools`, and not the platform, build-tools and NDK?

Those three packages must match exact versions written in the project's `android/build.gradle` (`compileSdkVersion`, `buildToolsVersion`, `ndkVersion`). The project does not exist yet in Milestone 1, and the versions on the React Native setup page can be older than what the template asks for. `platform-tools` has no such link to the project, and adb is needed right away to test the phone.

1.3 You edit `~/.zshrc` and the terminal you edited from still cannot find `adb`. Why?

A shell reads its config file only when it starts. The running terminal still has the environment it started with. A new terminal (or `source ~/.zshrc`) picks up the change.

1.4 `adb devices` shows `unauthorized`. Is this a Linux problem or a phone problem, and what fixes it?

It is the phone: Linux already let adb open the device, but the phone has not accepted this computer's key. Accept the prompt on the phone. If it does not appear, `adb kill-server` and reconnect.

1.5 Why install JDK 17 instead of the newest JDK?

React Native's setup page recommends JDK 17 and warns that you may run into problems with higher JDK versions. Using the recommended version removes one possible cause when a build fails.

Milestone 2: Project and editor

2.1 You change text in `App.tsx`. Why don't you need `npm run android` again?

Only the JavaScript half changed. Metro sends the new bundle to the app, and Fast Refresh swaps it in. A rebuild is only needed when native code or files in `android/` change.

2.2 What does `noUncheckedIndexedAccess` change about the type of `list[0]`, and why is that useful for TMDB data?

The type becomes "element or `undefined`", because the list might be empty. TMDB lists can be empty (a series without seasons, a search without results), so the compiler forces you to handle that case instead of crashing.

2.3 Why was the template's `App.test.tsx` deleted instead of kept?

It renders the whole app. Once the app uses native libraries such as op-sqlite and React Navigation, it can no longer run in Jest without many mocks, and this project tests logic, not screens.

2.4 What is the job of Metro, and what is the job of Gradle?

Metro bundles your TypeScript and serves it to the app during development. Gradle builds the native Android app from `android/` into an installable file.

Milestone 3: Your own types and progress logic

3.1 Why are anime not a separate `MediaType`?

TMDB lists anime as normal TV series with seasons and episodes, so they behave exactly like other series. A separate type would add cases without adding behaviour.

3.2 Why does `progressBadge` return an object with `kind` instead of just a number or `null`?

There are three different outcomes, and one of them carries a number. An object with `kind` names each outcome, and TypeScript only allows `value` where it exists. A bare number or `null` would hide the difference between "done" and "nothing to show".

3.3 What makes `progressPercent` a pure function, and why does that make it easy to test?

Its result depends only on its two parameters, and it changes nothing outside itself. A test only needs to call it with values and compare the result: no files, network, database or mocks.

3.4 Why does `latestEpisode` take a `readonly` array?

It promises not to change the caller's list. Finding the latest episode must not reorder the caller's data, and `readonly` makes an accidental `sort()` a compile error.

3.5 Why are specials (season 0) left out of the episode count?

Most people skip specials, and they are often added late or counted oddly. Counting them would stop many series from ever reaching 100%.

Milestone 4: Talking to TMDB

4.1 Why does `tmdbGet` return `unknown` instead of `any`?

`unknown` forces every caller to say what the data is before using it, so unchecked network data cannot spread through the app. `any` would turn off type checking for everything it touches.

4.2 Why test `map.ts` with saved fixtures instead of calling TMDB in the tests?

Fixtures make tests fast, repeatable and offline. Real network tests would fail when you have no internet, when TMDB is slow, or when TMDB changes popular data, without your code being wrong.

4.3 Why calculate `totalEpisodes` yourself instead of using a field from TMDB?

You control the rule: specials are excluded on purpose. A field from TMDB might include or exclude specials in a way you cannot see or change.

4.4 The token is git-ignored. Is it therefore secret on your phone? Explain.

No. Git-ignoring only keeps it out of the repository. The token is compiled into the app bundle on the phone, and anyone with the app file could extract it. That is why the app must not be published like this.

4.5 What should `searchTitles('  ')` do, and why?

Return an empty list without calling TMDB. A query of only spaces would at best return nothing useful and would waste a request against the rate limit.

Milestone 5: Designing the database

5.1 Why do the query functions take a `Db` parameter instead of importing op-sqlite?

So the same functions run against op-sqlite on the phone and better-sqlite3 in Jest. This is dependency injection: the caller decides which database to pass in.

5.2 Why is the progress percentage not stored in the `titles` table?

It can always be calculated from `watched_episodes` and `total_episodes`. A stored copy could get out of sync with the real data.

5.3 What does `user_version` protect you from when you release a new version with a changed table?

An installed app keeps its old database file when updated. `user_version` tells the app which migrations that file already has, so it runs only the new ones instead of failing or losing data.

5.4 Why must values go through `?` placeholders instead of being put into the SQL string?

Placeholders send values separately from the SQL text. Values with quotes (like `Grey's Anatomy`) cannot break the statement, and nobody can inject SQL through input.

5.5 Why does `getAppDb` cache a Promise and not the finished `Db`?

If two callers ask before the first open has finished, they both get the same Promise and wait for the same work. Caching only the finished `Db` would let both start opening and migrating.

Milestone 6: Screens and navigation

6.1 Why does Details receive `mediaType` and `tmdbId` instead of a whole `TrackedTitle`?

The parameters stay small, and Details always loads current data itself. A copied object could be out of date by the time it is shown, and the title might not even be in the database yet (from search).

6.2 What does the param list let TypeScript check that it could not check before?

That every `navigate` call uses an existing screen name with the right parameters, and that each screen reads only parameters it actually receives.

6.3 Why was a rebuild needed in step 6.1 but not in step 6.4?

Step 6.1 added libraries with native code and changed Android files, which only Gradle can build. Step 6.4 only changed TypeScript, which Metro and Fast Refresh handle.

6.4 In the stack Home → Search → Details, what does the back button show, and why?

Search. Back pops the top screen (Details), revealing the one below it in the stack.

Milestone 7: The main screen grid

7.1 Why does Home own the selected status instead of `StatusFilter`?

Home needs the value to load the right titles. Keeping it in Home and passing it down means there is one source of truth, and `StatusFilter` can be reused elsewhere.

7.2 Why is `useFocusEffect` needed, when `useEffect` would also run on start?

Home stays mounted under Search and Details, so a start-only effect would not run again when you come back. `useFocusEffect` runs each time the screen becomes visible.

7.3 What would happen without `useCallback` around the loading function?

A new function would be created on every render. `useFocusEffect` would see it as changed and run again, which sets state, renders again, and so on in a loop.

7.4 `ProgressBadge` returns `null` for some titles. Where was the decision made, and why there?

In `progressBadge` in `src/domain/progress.ts`. The rules live in one pure, tested function, and the component only draws the result.

7.5 Why does `FlatList` draw faster than mapping 300 cards inside a `ScrollView`?

FlatList draws only the items on and near the screen and reuses them while scrolling. A ScrollView draws all 300 cards and loads all their images at once.

Milestone 8: Search and add

8.1 What problem does debouncing solve, and why 400 ms instead of 0 or 3000?

It stops a request per key press, which is slow and wastes the rate limit. Around 400 ms is short enough to feel instant after a pause and long enough to cover normal typing gaps. 0 is no debounce, and 3000 feels broken.

8.2 Describe the race condition that the `cancelled` flag prevents.

You type "mat" and then "matrix". Both requests run. If the "mat" answer arrives last, it overwrites the "matrix" results. The flag marks the old request so its late answer is ignored.

8.3 Why does `describeTmdbError` take `unknown` instead of `TmdbError`?

Anything can be thrown: your `TmdbError`, a `TypeError` from `fetch`, or something else. Accepting `unknown` means the function handles every case and the caller can pass a caught error directly.

8.4 Why is adding done on Details and not directly from the search list?

You see what the title is before choosing, and adding needs the full details (the series' total episodes), which the search result does not contain.

8.5 Why does a movie count as 1 episode in `toNewTitle`?

A movie is one thing to watch. Storing 1 keeps the column meaningful and avoids dividing by zero anywhere, even though the movie badge never shows a percentage.

Milestone 9: Details and tracking

9.1 Why is the whole details page one `FlatList` instead of a `ScrollView` with a list inside?

A `FlatList` inside a `ScrollView` cannot know which items are visible, so it draws everything and scrolling conflicts. One `FlatList` with a header scrolls as one page and stays lazy.

9.2 Why turn the watched list into a `Set` of string keys?

Each row needs "is this watched?". A `Set` answers that instantly with `has`, while searching an array for each row is slower and more code. String keys make two references to the same episode equal.

9.3 Why is the progress text not kept in state?

It can always be computed from the watched list and the totals. Stored state could go stale after a mark or reload, and then the screen would disagree with the database.

9.4 Why does the app not move a series to Done automatically at 100%?

100% only means all episodes known today. A new season may be announced, and only you know whether you are finished with a show.

9.5 Why does `formatNextEpisode` take today's date as a parameter?

So tests give the same result on any day. If it read the clock itself, the expected values would change over time.

References

React Native

Page	URL
Set up your environment	https://reactnative.dev/docs/set-up-your-environment
Get started without a framework	https://reactnative.dev/docs/getting-started-without-a-framework
Running on device	https://reactnative.dev/docs/running-on-device
Using TypeScript	https://reactnative.dev/docs/typescript
View	https://reactnative.dev/docs/view
Flexbox	https://reactnative.dev/docs/flexbox
Style	https://reactnative.dev/docs/style
View style props	https://reactnative.dev/docs/view-style-props
Layout props (position)	https://reactnative.dev/docs/layout-props#position
Image	https://reactnative.dev/docs/image
Pressable	https://reactnative.dev/docs/pressable
TextInput	https://reactnative.dev/docs/textinput
FlatList	https://reactnative.dev/docs/flatlist
ScrollView	https://reactnative.dev/docs/scrollview
ActivityIndicator	https://reactnative.dev/docs/activityindicator
Images (static resources)	https://reactnative.dev/docs/images#static-image-resources
Community CLI	https://github.com/react-native-community/cli

Android and Arch Linux

Page	URL
Command-line tools download	https://developer.android.com/studio#command-tools
Command-line tools overview	https://developer.android.com/tools
Android CLI overview	https://developer.android.com/tools/agents/android-cli
Android CLI: sdk	https://developer.android.com/tools/agents/android-cli/commands/sdk
Android CLI: sdk install	https://developer.android.com/tools/agents/android-cli/commands/sdk_install
Android CLI: sdk list	https://developer.android.com/tools/agents/android-cli/commands/sdk_list
sdkmanager (deprecated, layout instructions)	https://developer.android.com/tools/sdkmanager
Environment variables	https://developer.android.com/tools/variables
React Native template (0.87-stable)	https://github.com/react-native-community/template/tree/0.87-stable
android-udev-rules	https://github.com/M0Rf30/android-udev-rules
Developer options	https://developer.android.com/studio/debug/dev-options
Arch Wiki: Android	https://wiki.archlinux.org/title/Android
Arch Wiki: Java	https://wiki.archlinux.org/title/Java

Editor

Page	URL
nvim-lspconfig	https://github.com/neovim/nvim-lspconfig
vtsls	https://github.com/yioneko/vtsls
conform.nvim	https://github.com/stevearc/conform.nvim
mason.nvim	https://github.com/mason-org/mason.nvim

TypeScript and JavaScript

Page	URL
Everyday Types	https://www.typescriptlang.org/docs/handbook/2/everyday-types.html
Narrowing	https://www.typescriptlang.org/docs/handbook/2/narrowing.html
Object types	https://www.typescriptlang.org/docs/handbook/2/objects.html
Utility types	https://www.typescriptlang.org/docs/handbook/utility-types.html
TSConfig: noUncheckedIndexedAccess	https://www.typescriptlang.org/tsconfig/#noUncheckedIndexedAccess
MDN: Using Fetch	https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
MDN: URLSearchParams	https://developer.mozilla.org/en-US/docs/Web/API/URLSearchParams
MDN: Array reduce	https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/reduce
MDN: Promise.all	https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/all
MDN: Set	https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set
MDN: setTimeout	https://developer.mozilla.org/en-US/docs/Web/API/Window/setTimeout

React

Page	URL
State: a component's memory	https://react.dev/learn/state-a-components-memory
Rendering lists	https://react.dev/learn/rendering-lists
Synchronizing with Effects	https://react.dev/learn/synchronizing-with-effects
Reusing Logic with Custom Hooks	https://react.dev/learn/reusing-logic-with-custom-hooks
Choosing the State Structure	https://react.dev/learn/choosing-the-state-structure
useCallback	https://react.dev/reference/react/useCallback

React Navigation

Page	URL
Getting started	https://reactnavigation.org/docs/getting-started
Hello React Navigation	https://reactnavigation.org/docs/hello-react-navigation
Moving between screens	https://reactnavigation.org/docs/navigating
Passing parameters	https://reactnavigation.org/docs/params
TypeScript	https://reactnavigation.org/docs/typescript
useFocusEffect	https://reactnavigation.org/docs/use-focus-effect

TMDB

Page	URL
API settings (get the token)	https://www.themoviedb.org/settings/api
Authentication	https://developer.themoviedb.org/docs/authentication-application
Image basics	https://developer.themoviedb.org/docs/image-basics
FAQ (attribution)	https://developer.themoviedb.org/docs/faq
Logos and attribution	https://www.themoviedb.org/about/logos-attribution
Rate limiting	https://developer.themoviedb.org/docs/rate-limiting
Search: Multi	https://developer.themoviedb.org/reference/search-multi
TV series details	https://developer.themoviedb.org/reference/tv-series-details
TV season details	https://developer.themoviedb.org/reference/tv-season-details
Movie details	https://developer.themoviedb.org/reference/movie-details

SQLite and databases

Page	URL
op-sqlite installation	https://op-engineering.github.io/op-sqlite/docs/installation
op-sqlite API	https://op-engineering.github.io/op-sqlite/docs/api
better-sqlite3 API	https://github.com/WiseLibs/better-sqlite3/blob/master/docs/api.md
SQLite CREATE TABLE	https://www.sqlite.org/lang_createtable.html
SQLite foreign keys	https://www.sqlite.org/foreignkeys.html
SQLite UPSERT	https://www.sqlite.org/lang_upsert.html
SQLite SELECT	https://www.sqlite.org/lang_select.html
SQLite core functions (COALESCE)	https://www.sqlite.org/lang_corefunc.html#coalesce
SQLite PRAGMA user_version	https://www.sqlite.org/pragma.html#pragma_user_version

Testing and tools

Page	URL
Jest: Getting Started	https://jestjs.io/docs/getting-started
Jest: Using Matchers	https://jestjs.io/docs/using-matchers
npm scripts	https://docs.npmjs.com/cli/using-npm/scripts
gitignore	https://git-scm.com/docs/gitignore
curl manual	https://curl.se/docs/manpage.html
Markdown tables	https://www.markdownguide.org/extended-syntax/#tables